

Image Classification

01

CNN 복습

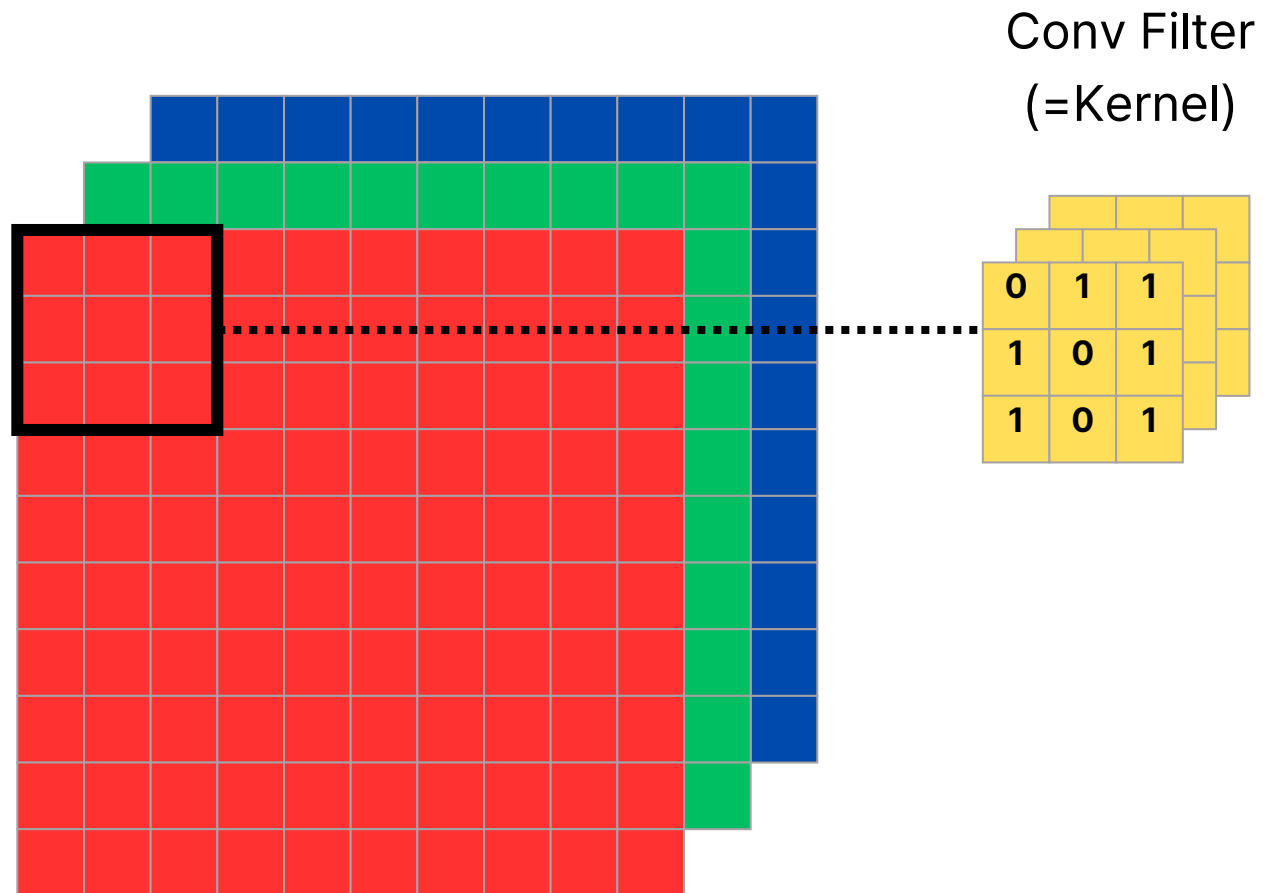
CNN (Convolutional Neural Network)

- 합성곱 신경망은 주로 이미지 인식과 같은 컴퓨터 비전 분야의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류
- 입력 데이터의 **지역적인 특징**을 추출하는데 특화된 구조를 갖고 있으며, 이를 위해 "**합성곱(Convolution)**" 연산을 사용
- 합성곱 신경망(CNN)은 시각 분야에 국한되지 않고 음성인식이나 자연어처리 같은 다른 작업에도 많이 사용되는 대표적인 신경망

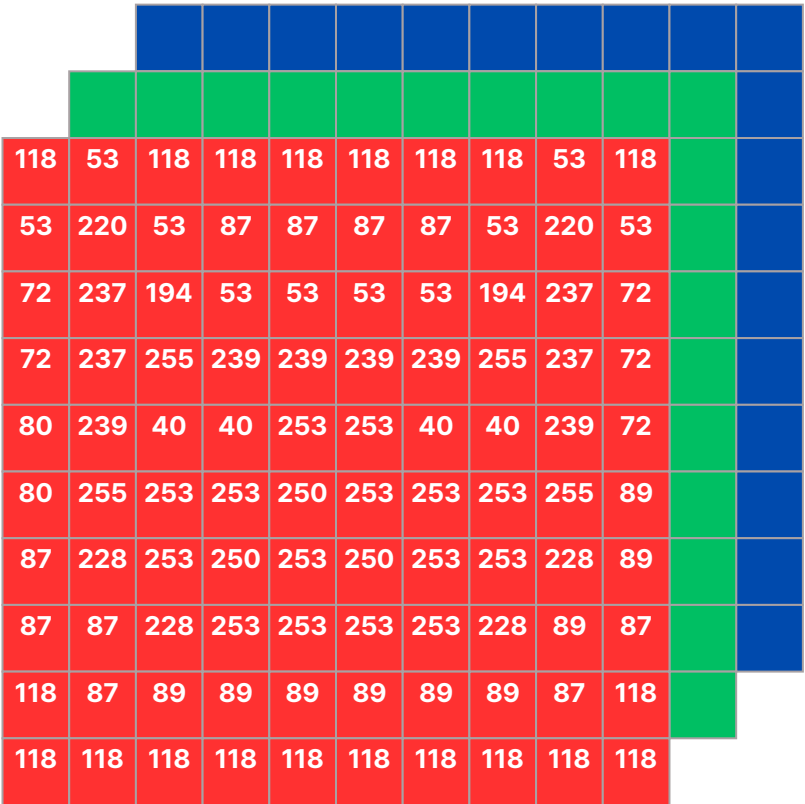
Convolutional Layer

컨볼루션 연산이 수행되는 레이어
지역적 특성을 추출하는 과정

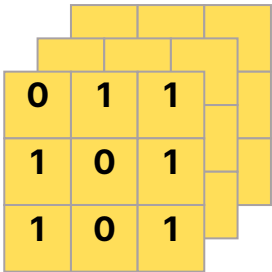
Convolution

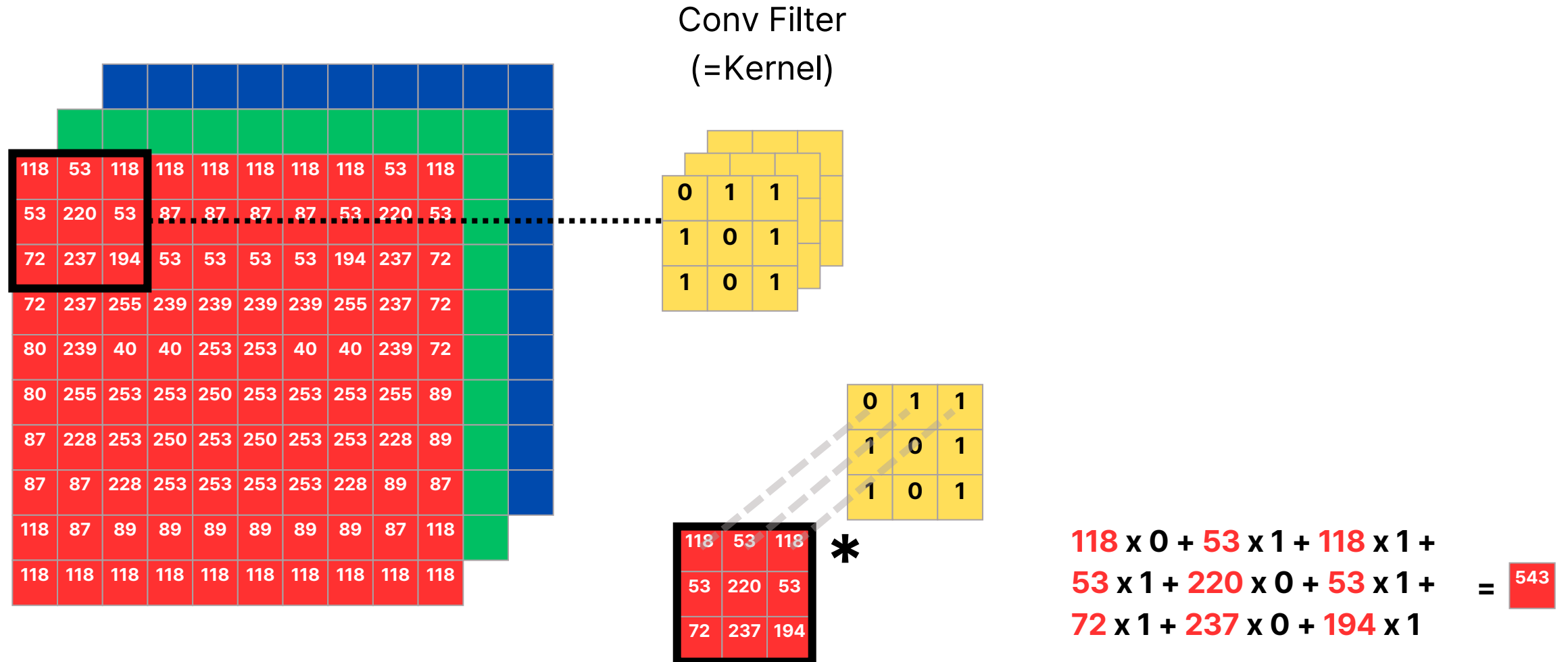


Convolution



Conv Filter
(=Kernel)





			0	1	1
			1	0	1
			1	0	1

118	53	118
53	220	53
72	237	194

*

$$118 \times 0 + 53 \times 1 + 118 \times 1 + 53 \times 1 + 220 \times 0 + 53 \times 1 + 72 \times 1 + 237 \times 0 + 194 \times 1$$

=

543

+

187	35	187
35	156	35
48	162	131

*

1	0	0
0	1	0
0	0	1

$$187 \times 1 + 35 \times 0 + 187 \times 0 + 35 \times 0 + 156 \times 1 + 35 \times 0 + 48 \times 0 + 162 \times 0 + 131 \times 1$$

+

474

+

3

=

1277

Bias

170	7	170
7	62	7
11	52	40

*

0	0	1
1	1	1
1	0	0

$$170 \times 0 + 7 \times 0 + 170 \times 1 + 7 \times 1 + 62 \times 1 + 7 \times 1 + 11 \times 1 + 52 \times 1 + 40 \times 0$$

257

입력값



출력값 = 특성 맵(Feature Map)

예: 외곽 선 특성 정보

1274	1378	1530	1532	1532	1593	1384	1220
1365	1668	1436	1389	1389	1409	1744	1294
1316	1679	1744	1640	1572	1754	1883	1284
1649	2069	2215	2676	2685	2322	2099	1526
1795	2341	2839	2857	2907	2694	2532	1770
2422	2819	3138	3273	3066	3217	2741	2251
1909	2298	2518	2467	2557	2419	2249	1729
1974	1755	1820	1845	1845	1806	1608	1863

출력값 = 특성 맵(Feature Map)

예: 외곽 선 특성 정보

1274	1378	1530	1532	1532	1593	1384	1220
1365	1668	1436	1389	1389	1409	1744	1294
1316	1679	1744	1640	1572	1754	1883	1284
1649	2069	2215	2676	2685	2322	2099	1526
1795	2341	2839	2857	2907	2694	2532	1770
2422	2819	3138	3273	3066	3217	2741	2251
1909	2298	2518	2467	2557	2419	2249	1729
1974	1755	1820	1845	1845	1806	1608	1863

(1,1) 주변 3x3 영역에서 지역적인 특성을 수치화

(1,2) 주변 3x3 영역에서 지역적인 특성을 수치화

...

Strided Convolution

Input Image

1	9	8	4	4	5	7
4	8	6	7	9	1	7
4	0	5	9	3	8	4
7	3	5	9	0	5	4
7	4	1	1	8	1	2
7	6	6	9	8	7	6
3	6	3	5	4	2	7

Kernel

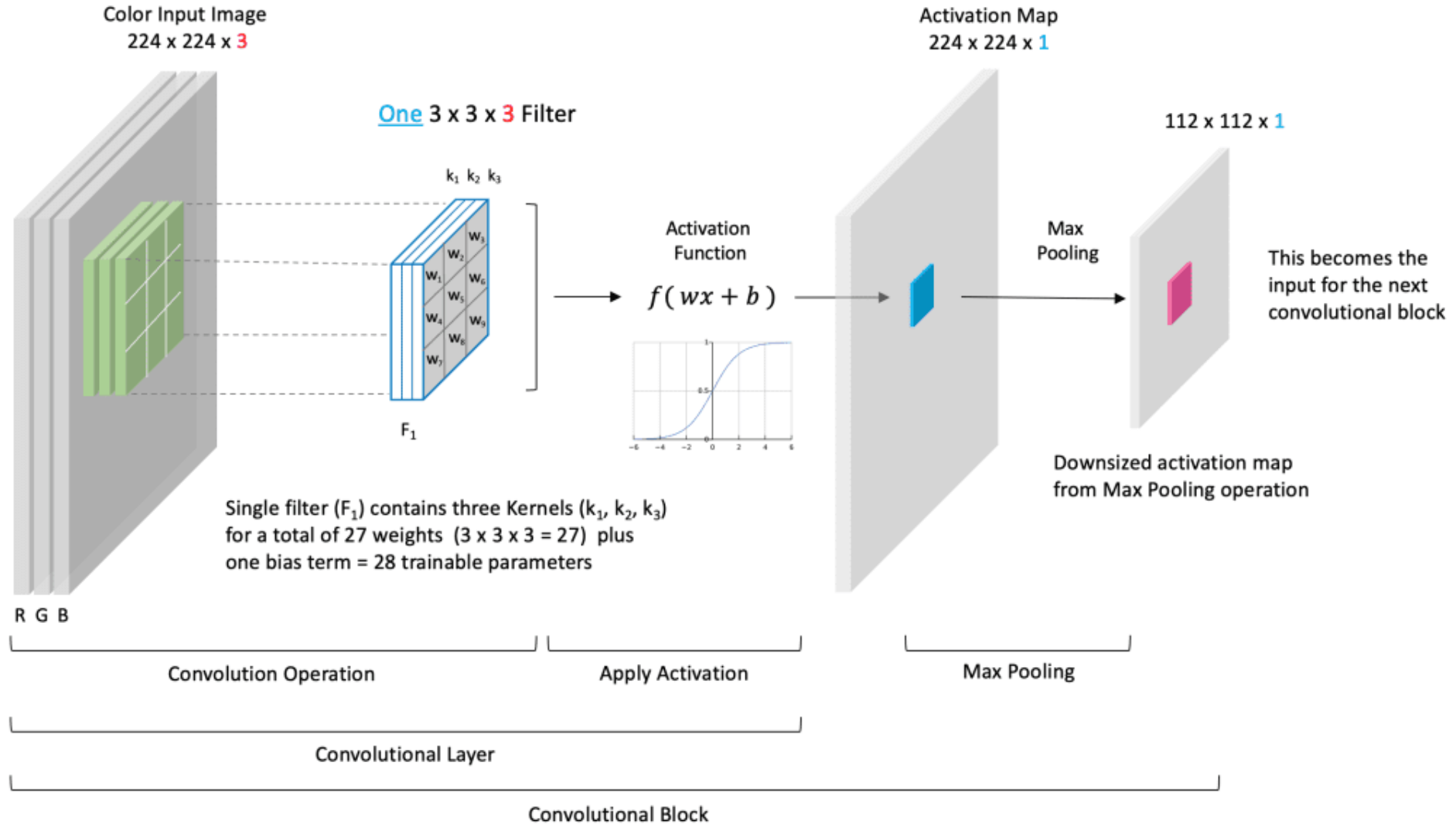
0	-1	0
-1	5	-1
0	-1	0

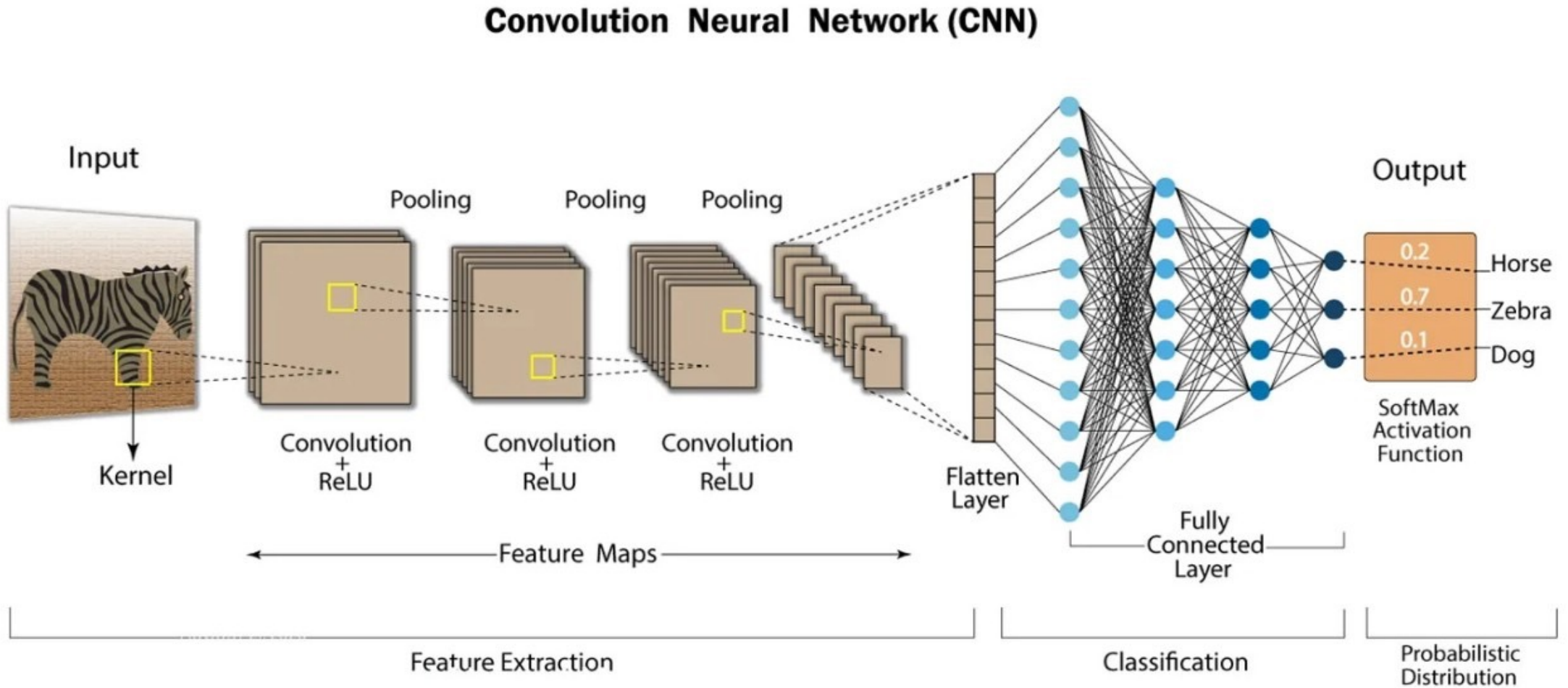
*

=

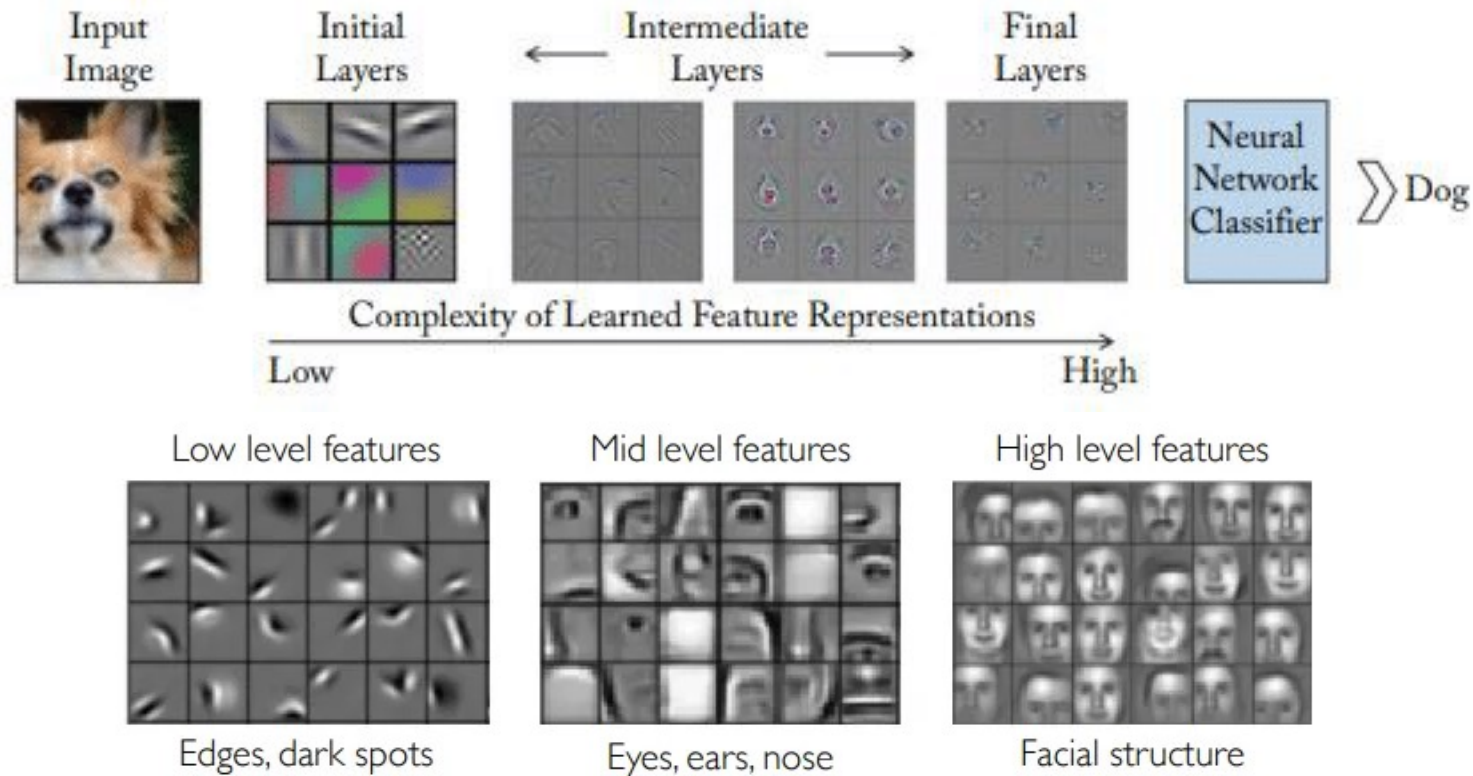
Feature Map

이 과정의 핵심은
바로 "Kernel"





- Convolutional Neural Network(CNN)은 시각 피질의 작동 방식에서 영감을 받은 인공 신경망
- 이미지를 처리할 때 특징을 추출하는 작은 필터를 이용하여 이미지 전체를 스캔

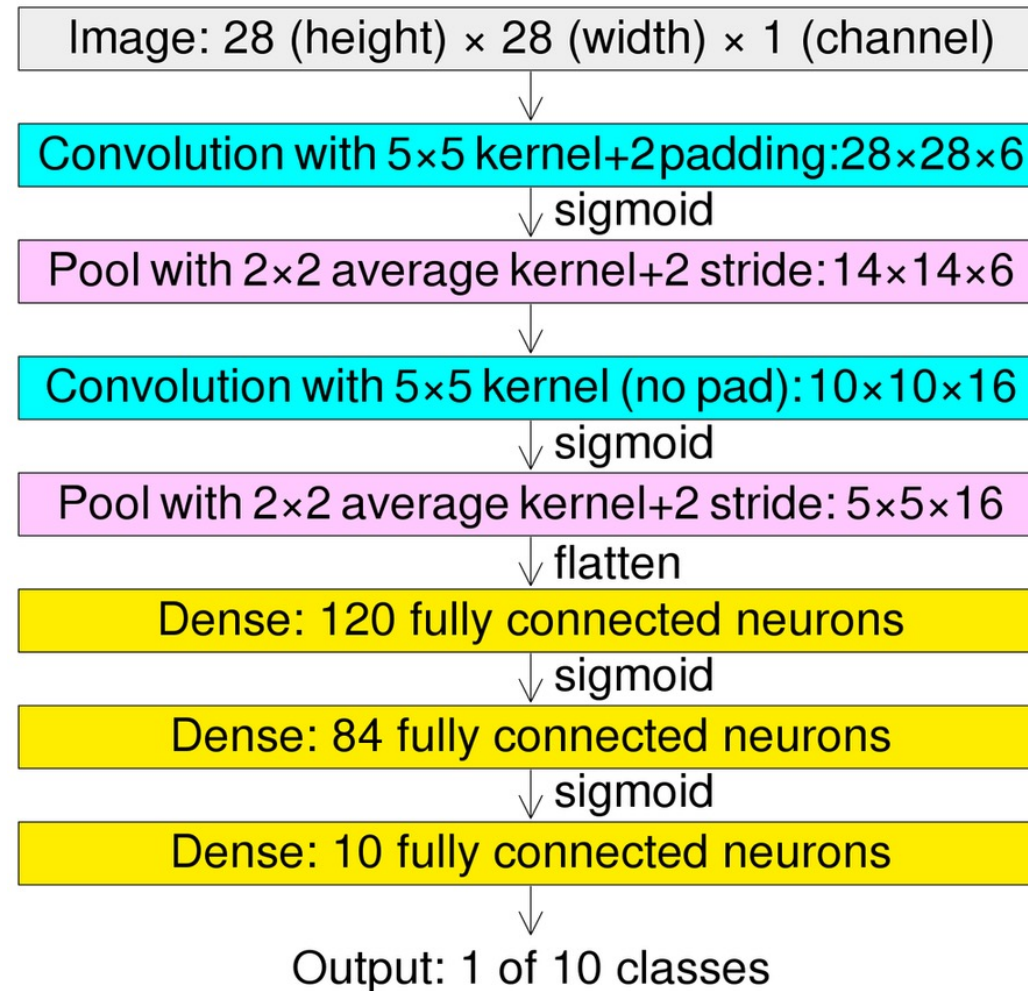


대표적 CNN 모델

LeNet

Lecun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. (1998).

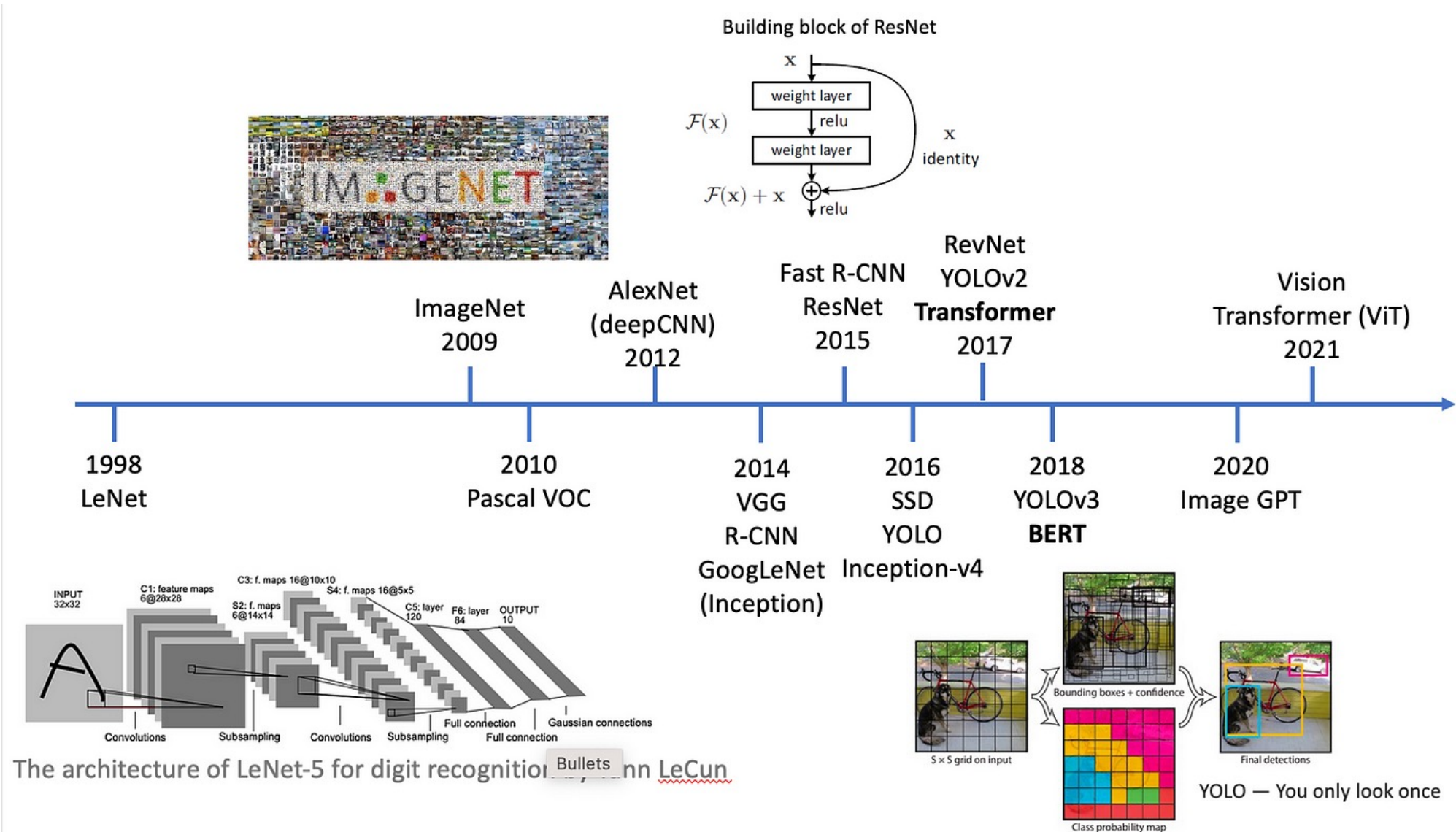
"Gradient-based learning applied to **document recognition**"

LeNet

LeNet

- 딥 러닝 > 다른 머신 러닝
- 이미지 인식에 효과적 (98% 정확도)
- 딥러닝 여러 연구에서 벤치마크로 활용되는 데이터셋 제시

A brief history of deep learning models in computer vision



학습 방법

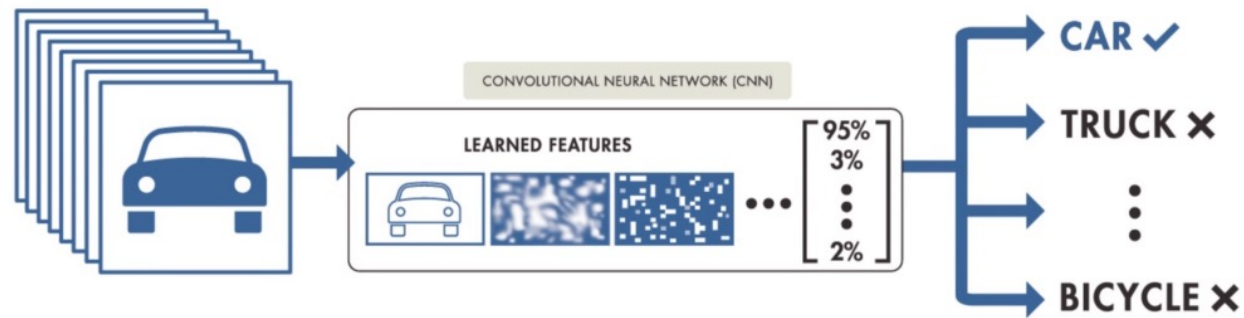
처음부터 학습
사전 학습된 모델의 활용

모델 구조

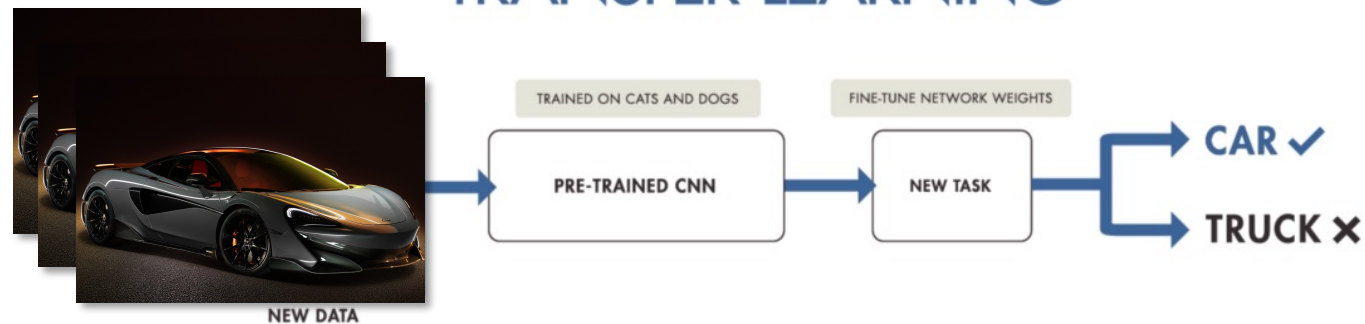
LeNet
AlexNet
VGGNet
Inception
ResNet
DenseNet
SeNet
MobileNet
EfficientNet
etc

1000 종류의 탈 것을 분류하는 모델이 이미 있다.
이 모델을 10가지의 스포츠카 종류를 분류하는 새로운 테스트에 활용할 수 있을까?

TRAINING FROM SCRATCH



TRANSFER LEARNING

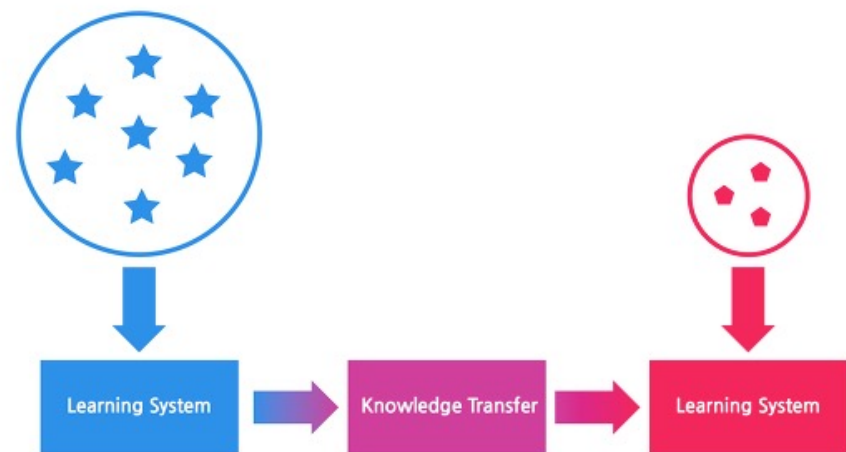


전이학습

어떤 작업을 수행하기 위해 이미 사전 학습된 모델을 재사용해 새로운 작업이나 관련 도메인(Domain)의 성능을 향상시킬 수 있는 기술

- 전이학습은 기존에 학습된 모델의 지식(Weights, Features)을 새로운 문제에 활용하는 딥러닝 기술입니다.
- 기존 모델은 대규모 데이터셋(예: ImageNet)에서 사전 학습(Pretraining)되어 일반적인 특징(Edges, Shapes, Patterns)을 학습한 상태입니다.
- 전이학습은 이를 기반으로 새로운 데이터셋에서 빠르게 학습하거나 성능을 개선하는 방식입니다.

Transfer learning



전이학습의 필요성

1. 데이터 부족 문제 해결

- 딥러닝 모델은 대량의 데이터를 요구하지만, 많은 경우 데이터 수집이 어렵거나 비용이 높습니다.
- 전이학습은 대규모 데이터에서 학습된 모델을 사용하여 적은 데이터로도 높은 성능을 달성할 수 있습니다.

2. 학습 시간 단축

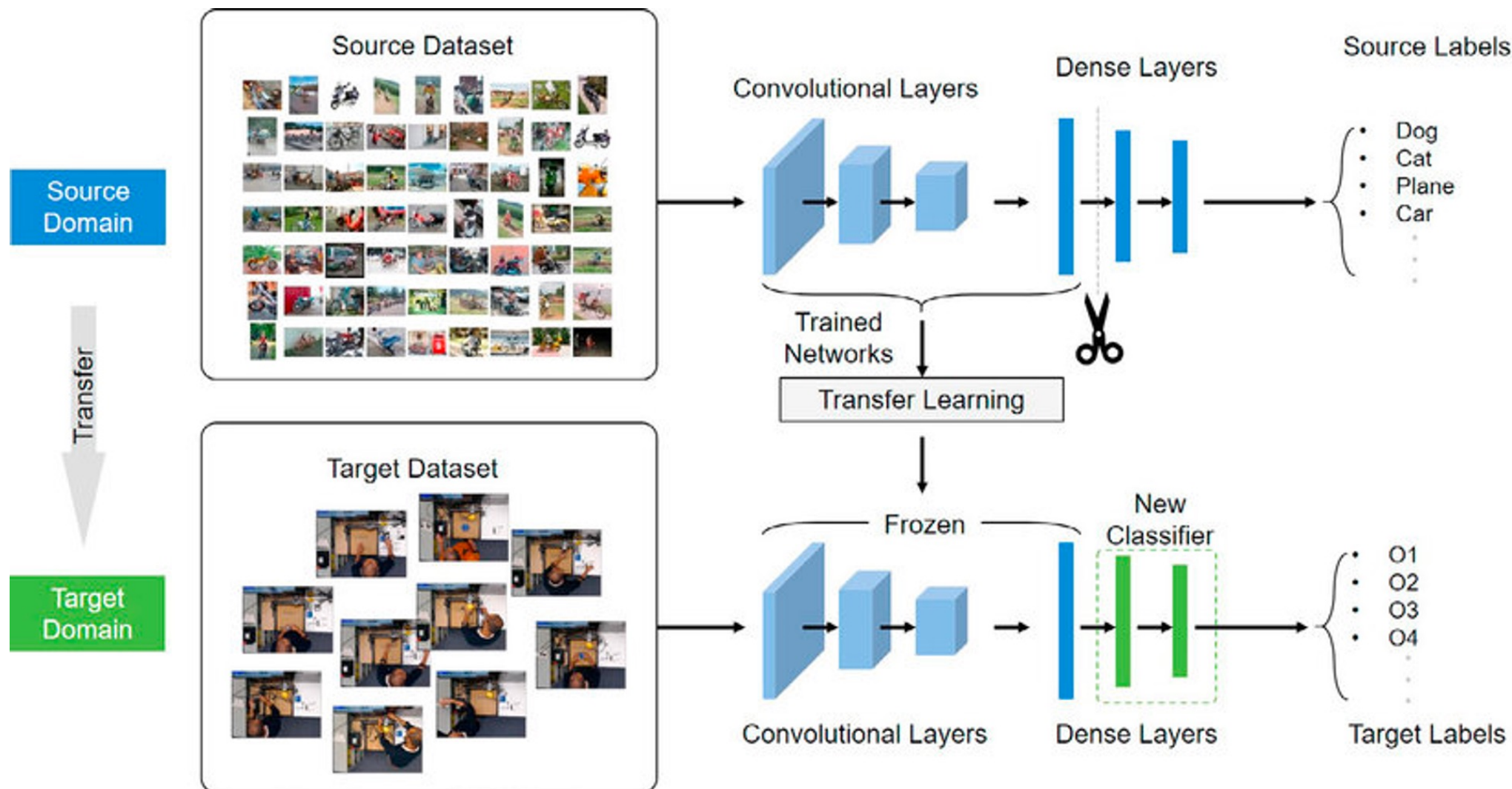
- 처음부터 모델을 학습하는 대신, 사전 학습된 모델의 가중치를 활용하면 학습 시간을 크게 줄일 수 있습니다.
- 특히, 네트워크 구조가 크고 복잡할수록 전이학습이 효과적입니다.

3. 성능 개선

- 사전 학습된 모델은 다양한 데이터에서 일반적인 특징을 이미 학습한 상태이므로, 새로운 문제에서도 성능 향상에 도움을 줍니다.
- 예를 들어, 자연 이미지에서 학습한 모델은 다양한 물체의 기본적인 특징(Edges, Shapes)을 학습한 상태로, 이를 활용해 빠르게 적응합니다.

4. 효율적인 자원 활용

- 대규모 데이터셋에서 모델을 학습하려면 고성능 GPU가 필요하지만, 전이학습은 상대적으로 적은 자원으로도 높은 성능을 제공합니다.



Feature Extraction (특징 추출)

- 사전학습된 모델의 가중치를 고정(freeze)하고, 마지막 분류기(fully connected layer)만 새 데이터셋에 맞게 재학습하는 방법
- 장점: 빠른 학습, 과적합 위험 감소
- 단점: 사전학습된 특징에 의존하므로 표현력 제한 가능
- 데이터의 양이 적을 때 주로 사용

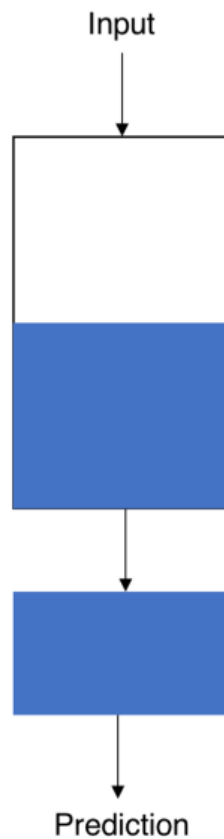
Fine-tuning (미세조정)

- 사전학습된 모델의 일부 또는 전체 가중치를 새로운 데이터셋에 맞게 조정하는 방법
- 장점: 모델을 문제에 최적화 가능
- 단점: 학습 시간이 길어지고 과적합 위험이 있을 수 있음
- 데이터가 충분하고 세밀한 조정이 필요시 주로 사용

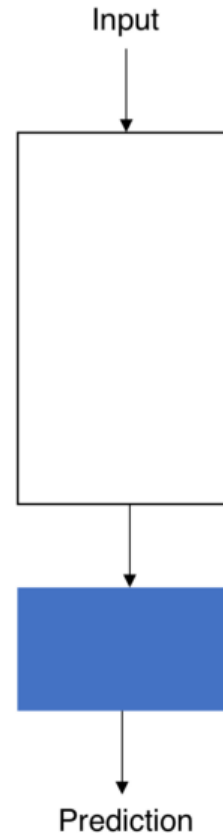
Strategy 1
Train the
entire model



Strategy 2
Train some layers and
leave the others frozen



Strategy 3
Freeze the
convolutional base



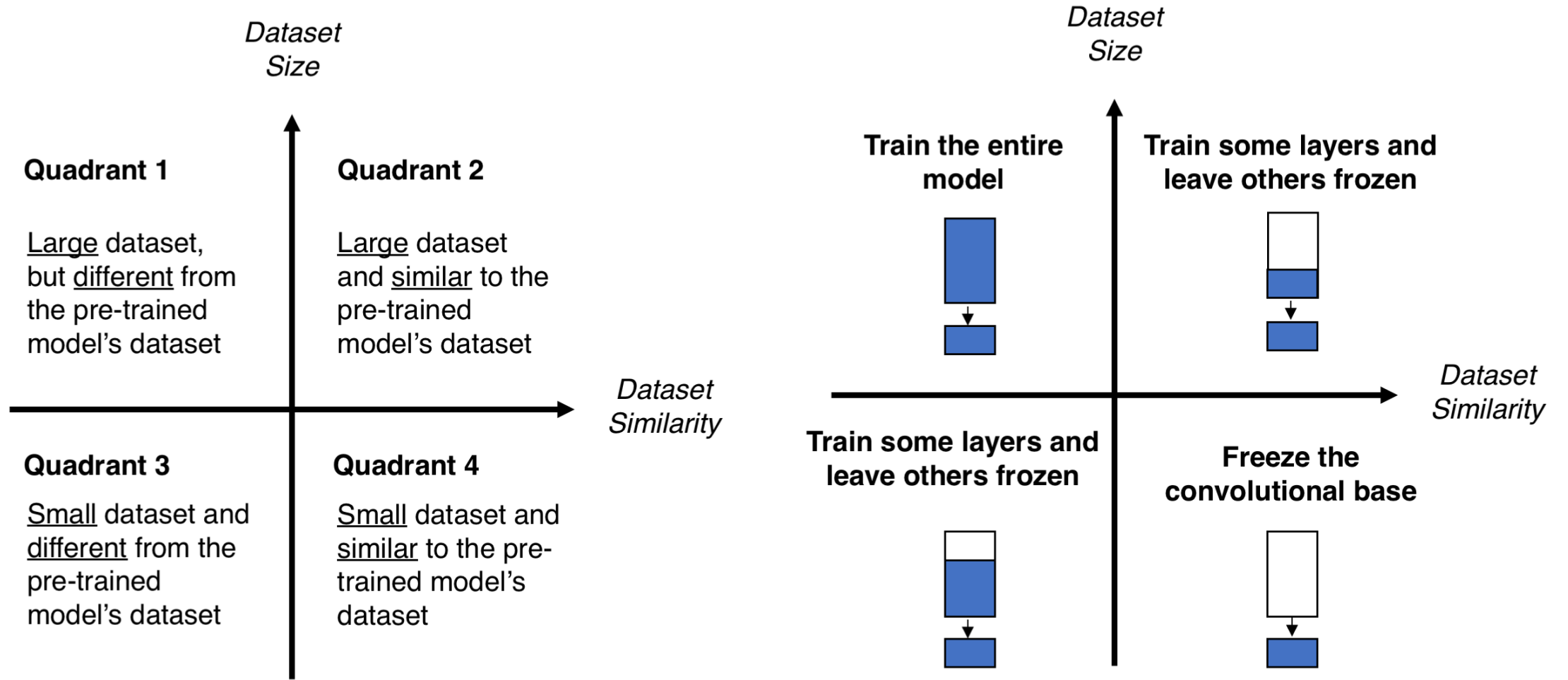
Legend:



Frozen



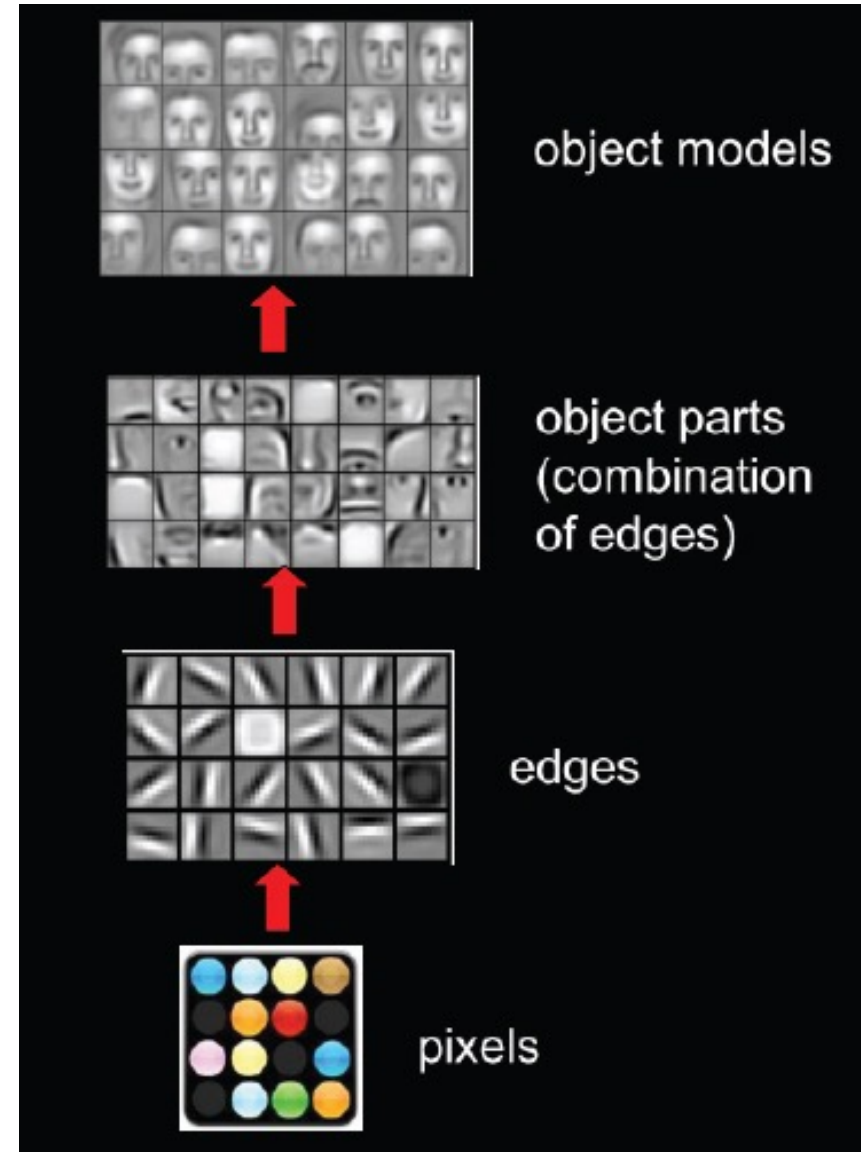
Trained



전이학습을 잘 사용하면 작은 데이터셋에 대해서도
"성능이 좋은" 모델을 "빠르게 " 만들 수 있다.

그렇다면 전이학습이 가능한 이유는 무엇일까?

Low Level Features!



Task A에서 Task B로 모델을 전이학습시킨다고 할 때,

1. Task A와 Task B가 같은 입력을 가질 때
Task A and B have the same input x

2. Task A가 B보다 훨씬 더 많은 데이터를 가지고 있을 때
You have a lot more data for Task A than Task B

3. A의 낮은 수준 특성들이 B를 학습하는 데 도움이 되는 경우
Low level features from A could be helpful for learning B

1. 장점

- 효율성: 데이터와 자원이 부족한 상황에서도 높은 성능을 제공
- 빠른 학습: 모델 초기화 상태에서 학습하지 않아 학습 시간이 단축
- 일반화 성능: 다양한 데이터에서 학습된 특징을 활용하여 일반화 능력을 강화

2. 한계

- 도메인 차이 문제:
사전 학습 데이터셋과 새 데이터셋이 크게 다를 경우, 성능이 저하될 수 있음
- 고정된 가중치의 한계:
Feature Extraction 방식에서는 초반 계층이 고정되므로 새로운 데이터셋에 완전히 적응하지 못할 수 있음
- 데이터 의존성:
사전 학습된 모델의 성능은 학습된 데이터셋의 크기와 품질에 따라 달라질 수 있음

1. 사전학습된 모델 선택

- 예: VGGNet, ResNet, DenseNet 등
- 선택 기준: 문제 특성, 모델의 복잡도, 리소스 제약 등

2. 모델 로드 및 분석

- 모델의 구조(특히, convolutional 부분과 분류기 부분)를 이해하고, 어느 부분을 재학습할지 결정

3. 분류기 수정

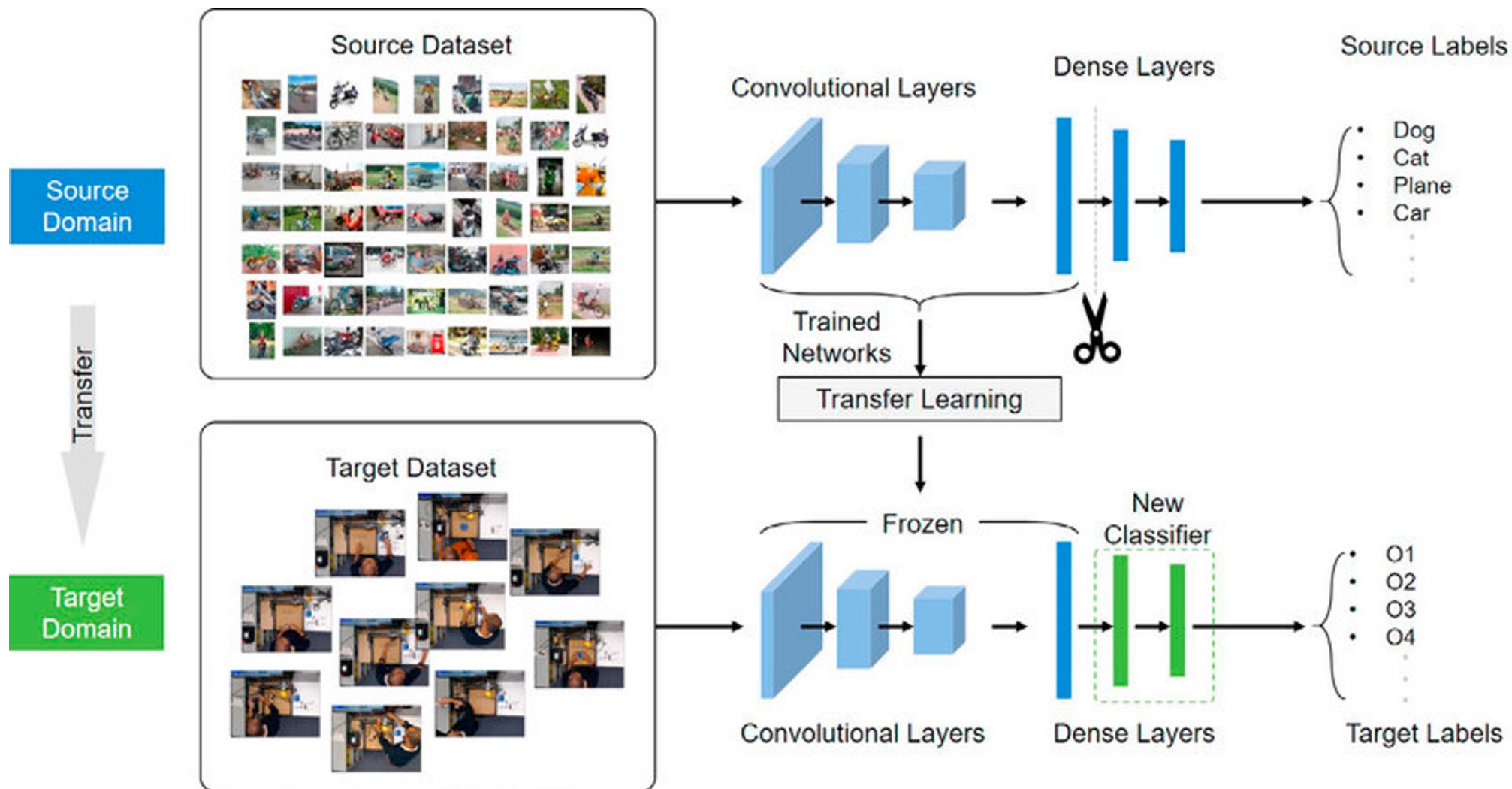
- 기존 분류기를 새로운 데이터셋의 클래스 수에 맞게 재설계
- 예: ImageNet의 1000 클래스 → 새로운 작업의 클래스 수 (예: 10)

4. 가중치 동결/해제 결정

- Feature Extraction: 사전학습된 계층은 동결, 마지막 분류기만 학습
- Fine-tuning: 일부 계층(보통 상위 계층)만 동결하고 나머지 계층을 미세조정

5. 모델 학습 및 평가

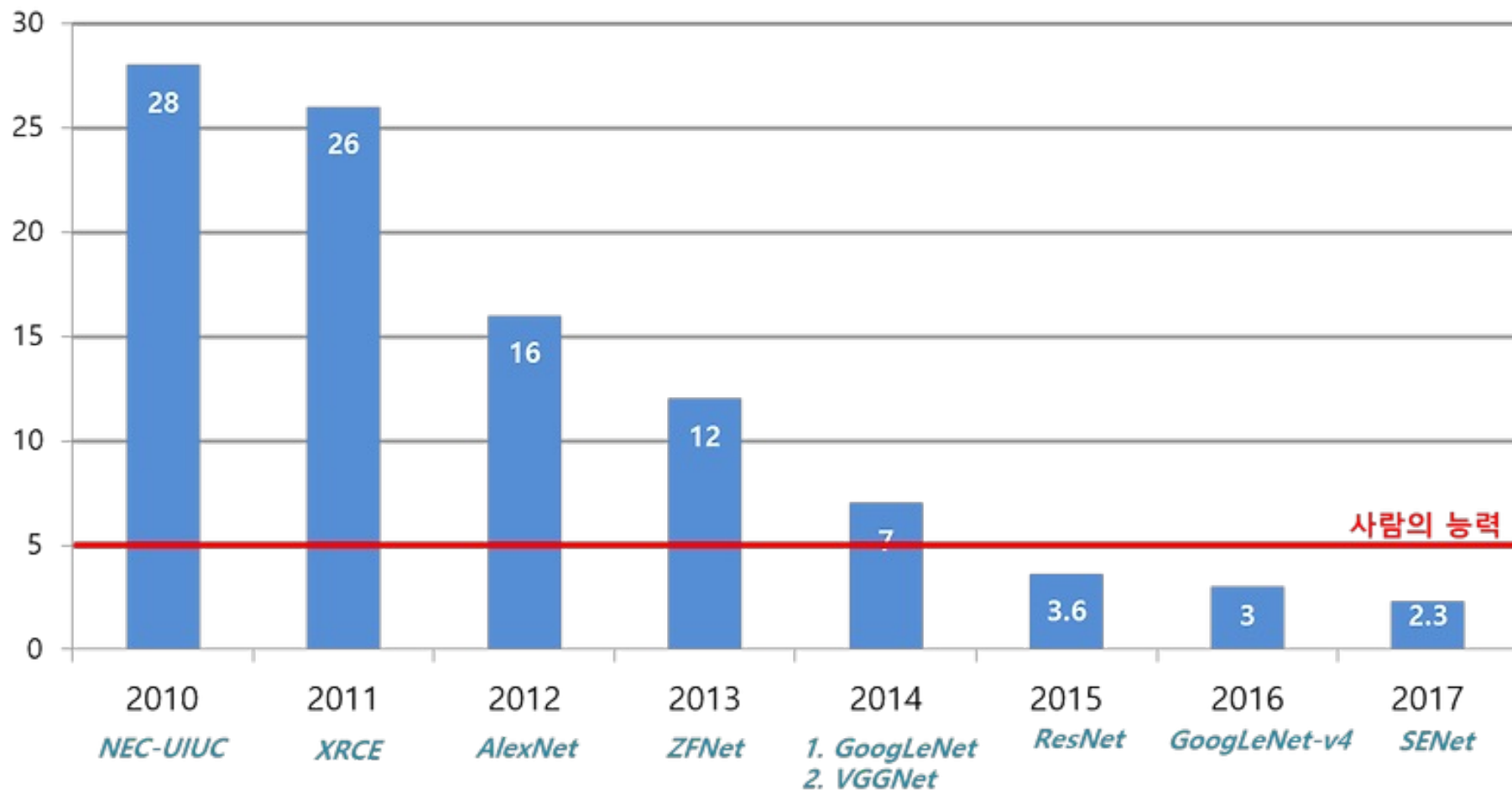
- 새로운 데이터셋에 대해 학습을 진행하고, 검증 및 테스트를 통해 성능 평가



ImageNet

- 대회 개요:
ImageNet 대회(ILSVRC)는 수백만 장의 이미지와 1000여 개의 클래스(범주)를 다루는 대규모 이미지 분류 및 객체 검출 경진대회입니다.
- ImageNet 프로젝트는 시각적 개체 인식 소프트웨어 연구에 사용하도록 설계된 대규모 시각적 데이터베이스로, 2023년 기준으로 약 1,420만 개의 이미지와 20,000개 이상의 카테고리 분류를 제공합니다)
- 의의:
 - 규모: 방대한 데이터셋을 통해 딥러닝 모델의 일반화 능력을 시험합니다.
 - 영향: 기존의 컴퓨터 비전 기법에 비해 딥러닝 모델(특히 AlexNet)의 성능 향상을 통해 이후 연구 및 산업 전반에 큰 파급 효과를 주었습니다.
 - <https://www.image-net.org/>

우승 알고리즘의 분류 에러율(%)



AlexNet

- 딥러닝 혁명의 시작 : AlexNet은 2012년 ImageNet 대회(ILSVRC)에서 우승한 합성곱 신경망 구조의 모델
- 2012년 이전에는 전통적인 머신러닝 모델(SVM, Random Forest 등)이나 얇은 구조로 인식 오류율이 26%에 달했지만, 알렉스넷은 인식 오류율을 약 16%까지 낮췄다.
- 알렉스넷의 등장 이후로 합성곱 신경망 모델이 인기를 끌기 시작했고, 이후부터 깊은 구조를 가진 모델들이 나타나 우승을 차지하기 시작했다.
- 주요 기여
 - GPU를 활용하여 대규모 데이터를 학습하는 기술을 도입
 - ReLU 활성화 함수와 Dropout 정규화로 학습 속도 및 과적합 문제 해결
 - 데이터 증강(Data Augmentation)을 모델 학습에 효과적으로 사용
 - https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

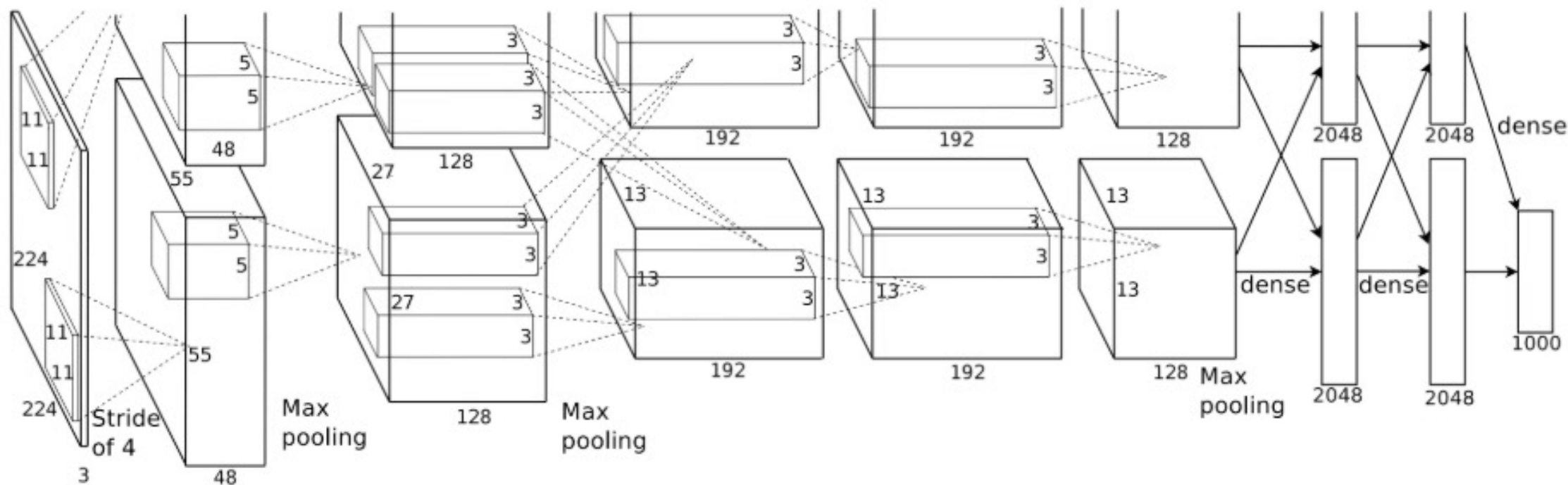


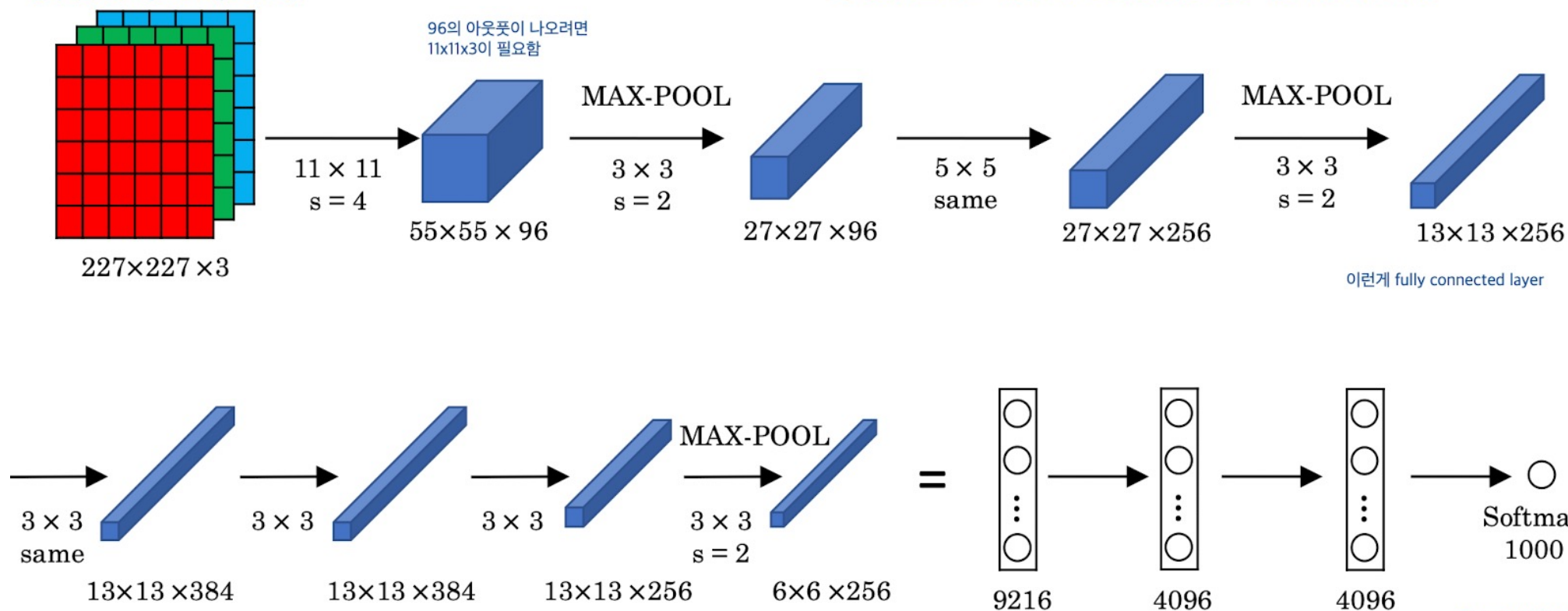
Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet

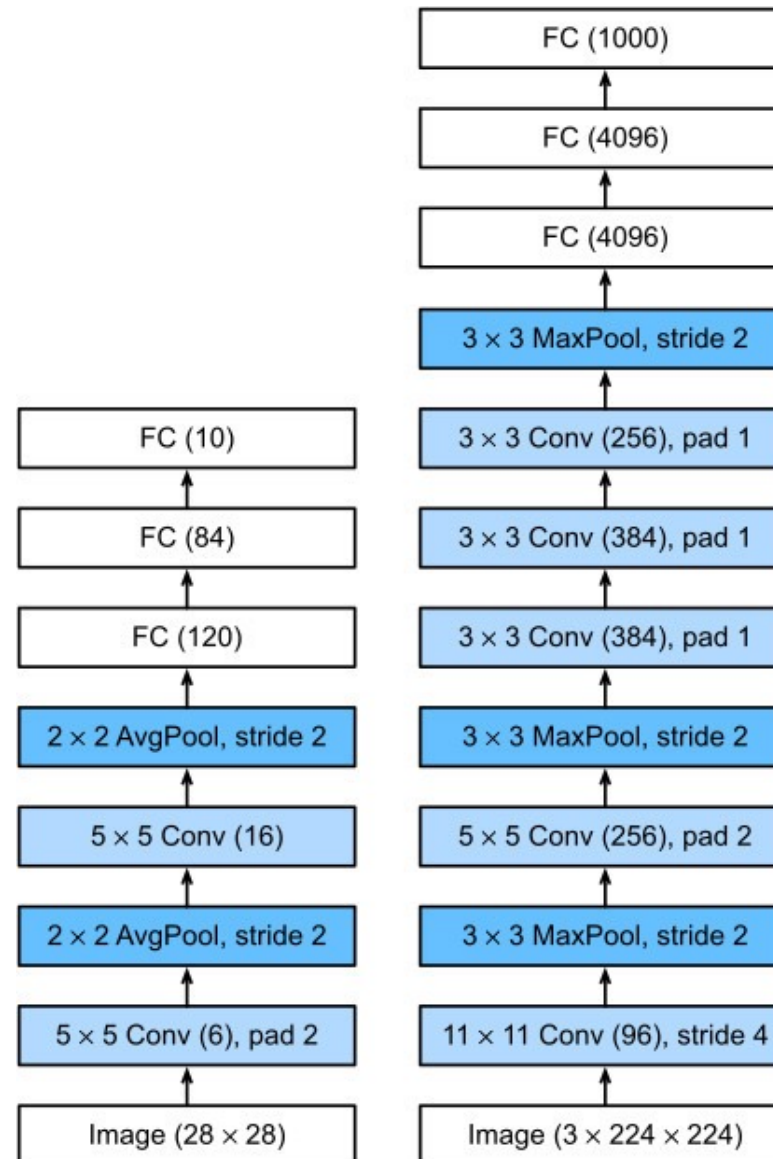
컬러 이미지에 사용됨

Architecture:
 CONV1
 MAX POOL1
 NORM1
 CONV2
 MAX POOL2
 NORM2
 CONV3
 CONV4
 CONV5
 Max POOL3
 FC6
 FC7
 FC8

컬러 이미지가 large conv layer에 들어감

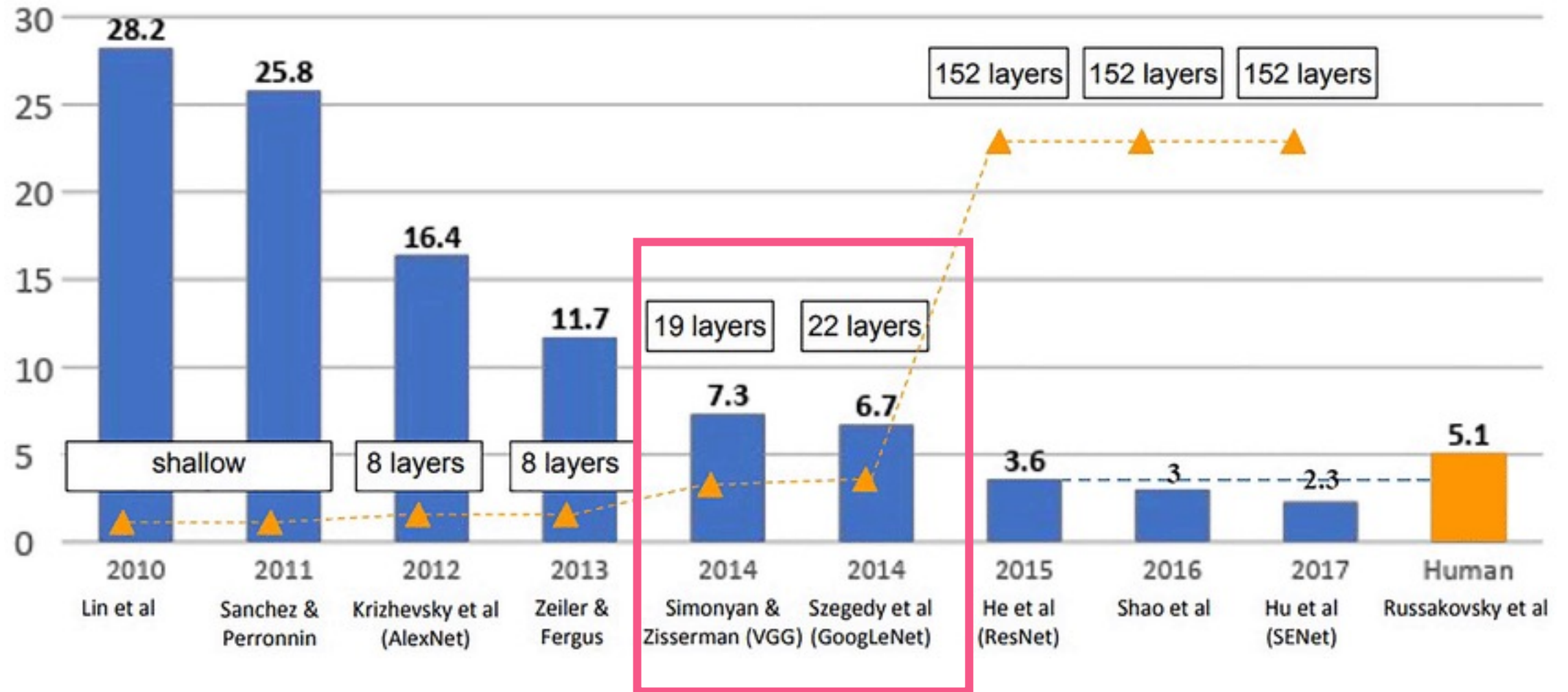


Output이 1000이 됨



Layer	Input	Filters	Output
CONV1	224×224×3	96 (11×11 filters, stride 4, pad 1. 5)	55×55×96
POOL1	55×55×96	3×3 filters, stride 2	27×27×96
NORM1	27×27×96	Normalization layer	27×27×96
CONV2	27×27×96	256 (5×5 filters, stride 1, pad 2)	
POOL2		3×3 filters, stride 2	
NORM2		Normalization layer	
CONV3		384 (3×3 filters, stride 1, pad 1)	
CONV4		384 (3×3 filters, stride 1, pad 1)	
CONV5		256 (3×3 filters, stride 1, pad 1)	
POOL3		3×3 filters, stride 2	
FC6		Fully-connected	
FC7		Fully-connected	
FC8		Fully-connected	1000

1. 최초의 CNN기반 ILSVRC(2012) 우승 모델
2. CNN에서 ReLU 활성화 함수를 최초로 대규모 적용
 - 기존 Sigmoid, Tanh 함수 대신 CNN에 ReLU(Rectified Linear Unit)를 최초로 대규모 적용
 - 계산량 감소, 학습 속도 증가, Gradient Vanishing 문제 완화.
3. 데이터 증강
 - 이미지 변환(translation) 및 수평 반전(horizontal reflection), RGB 채널 강도 조절
4. Multiple GPU 활용
 - NVIDIA GPU(CUDA)를 활용하여 대규모 데이터 학습을 가속화
 - 당시 CPU 기반 모델과 비교하여 10배 이상의 학습 속도 향상
5. 학습 파라미터
 - Dropout 0.5, Batch size 128, Optimizer : SGD + Momentum(0.9)
 - 초기 학습률 0.01, 검증 정확도가 정체되면 1/10로 수동 감소
 - L2 가중치 감소(weight decay) $5e-4$



VGG Net

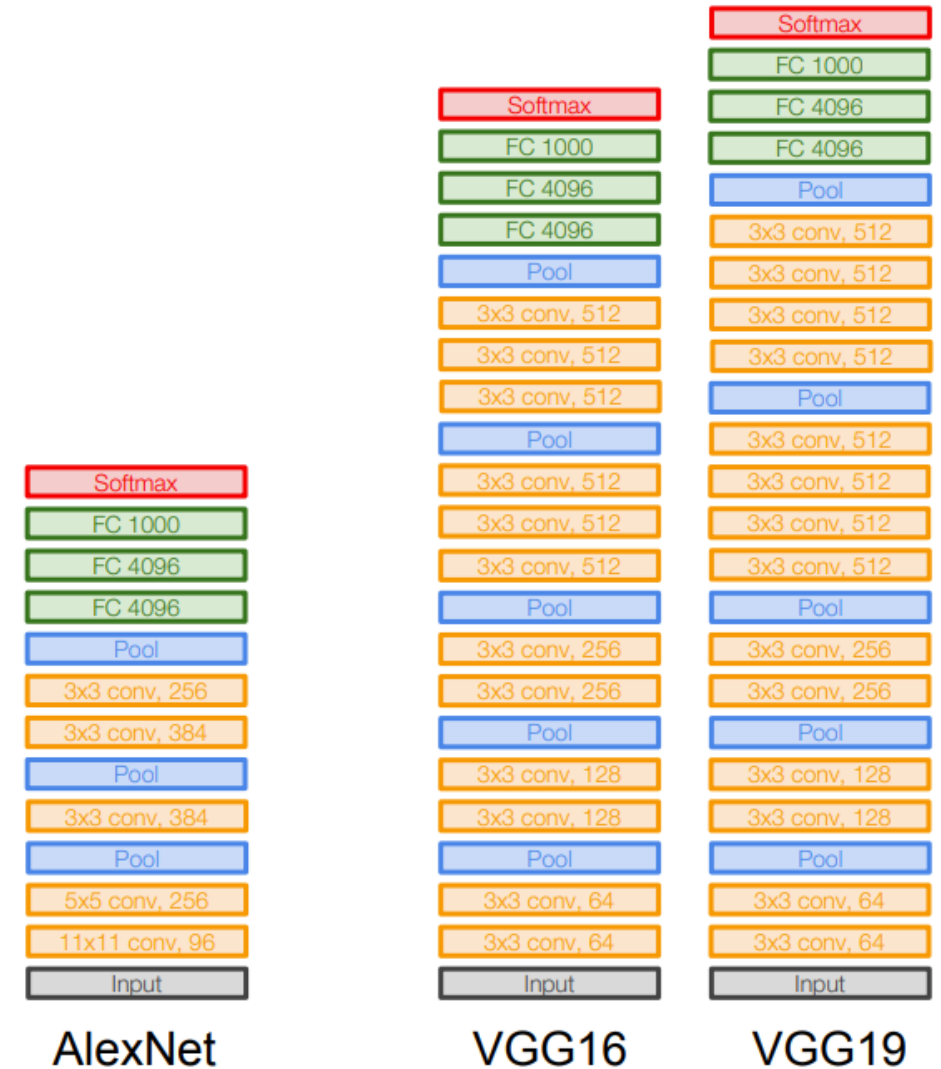
- VGGNet은 2014년 ILSVRC에서 발표된 모델로, 객체 위치 추정1위, 이미지 분류 분야에서 2위를 차지하였습니다
- 영국의 옥스퍼드 대학 Visual Geometry Group(VGG)에서 개발하였으며, 깊은 컨볼루션 신경망(CNN)의 효과를 극명하게 보여주며 큰 주목을 받았습니다.
- VGGNet은 단순하면서도 효과적인 구조 덕분에 이후 많은 연구에서 기초 모델로 사용되었습니다.
- 주요 기여
 - 단순한 구조로 딥러닝 모델의 설계와 분석을 쉽게 함.
 - CNN의 성능은 네트워크 깊이에 따라 향상될 수 있음을 입증.
 - 작은 3×3 필터의 반복 사용으로 연산 효율성과 성능 개선.
- "Very Deep Convolutional Networks for Large-Scale Image Recognition"
<https://arxiv.org/pdf/1409.1556>

VGGNet VS AlexNet

1. 깊이가 성능에 미치는 영향

논문에서는 네트워크의 깊이를 점진적으로 늘렸을 때 분류 성능이 크게 향상됨을 실험적으로 보여줍니다.

- 알렉스넷과 VGG-16 모두 이미지넷 데이터로 학습
 - 알렉스넷 : 약 120만 개의 이미지
 - VGG-16 : 약 1400만 개의 이미지
- AlexNet보다 VGG-16이 층 수가 약 2배
- 합성곱, ReLU, 풀링, FCN 사용한 건 동일하나, 필터 크기가 다름



VGGNet VS AlexNet

2. 작은 컨볼루션 필터의 활용:

VGGNet의 가장 큰 특징 중 하나는 3×3 크기의 필터를 일관되게 사용하는 것입니다.

하나의 5×5 또는 7×7 필터 대신 여러 개의 3×3 필터를 연속으로 쌓음으로써,

- 리셉티브 필드(Receptive Field): 예를 들어, 두 개의 연속된 3×3 컨볼루션은 5×5 필터와 동일한 리셉티브 필드를 가지지만, 두 번의 비선형 활성화(ReLU)를 포함하게 되어 더욱 복잡한 함수 근사를 가능하게 합니다.
- 파라미터 효율성: 작은 필터를 사용하면 동일한 리셉티브 필드를 얻으면서도 파라미터 수를 줄일 수 있습니다.

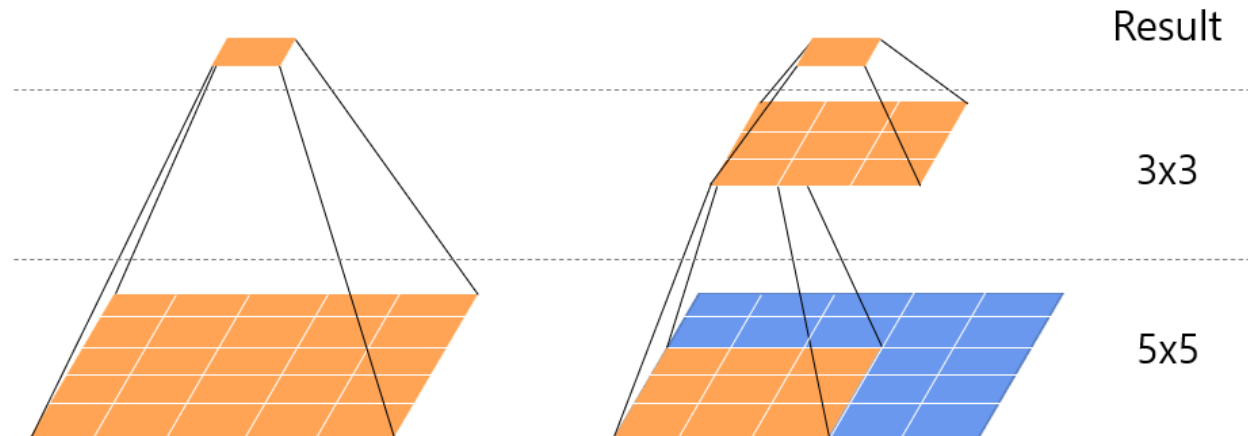


표 1: ConvNet 구성(열로 표시).

- 왼쪽(A)에서 오른쪽(E)으로 갈수록 더 많은 레이어가 추가되어 구성의 깊이가 깊어지며, 새로 추가된 레이어는 **볼드체**로 표시되어 있습니다.
- 합성곱 레이어의 파라미터는 “conv(수용영역 크기)-(채널 수)”로 표기합니다.
- 간결성을 위해 ReLU 활성화 함수 표기는 생략했습니다.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

계층	입력 크기	커널 크기(out C, in C, h, w)	출력 크기	설명
Conv1-1, Conv1-2	224×224×3	(3, 64, 3, 3) (64, 64, 3, 3)	224×224×64	3×3 필터, stride=1, padding=1
MaxPool1	224×224×64	2x2	112×112×64	2×2 풀링, stride=2
Conv2-1, Conv2-2	112×112×64	(128, 64, 3, 3) (128, 128, 3, 3)	112×112×128	3×3 필터, stride=1, padding=1
MaxPool2	112×112×128	2x2	56×56×128	2×2 풀링, stride=2
Conv3-1, Conv3-2, Conv3-3	56×56×128	(256, 128, 3, 3) (256, 256, 3, 3) (256, 256, 3, 3)	56×56×256	3×3 필터, stride=1, padding=1
MaxPool3	56×56×256	2x2	28×28×256	2×2 풀링, stride=2
Conv4-1, Conv4-2, Conv4-3	28×28×256	(512, 256, 3, 3) (512, 512, 3, 3) (512, 512, 3, 3)	28×28×512	3×3 필터, stride=1, padding=1
MaxPool4	28×28×512	2x2	14×14×512	2×2 풀링, stride=2
Conv5-1, Conv5-2, Conv5-3	14×14×512	(512, 512, 3, 3) (512, 512, 3, 3) (512, 512, 3, 3)	14×14×512	3×3 필터, stride=1, padding=1
MaxPool5	14×14×512	2x2	7×7×512	2×2 풀링, stride=2
Flatten	7×7×512	-	1×25088	Flatten 계층
FC1	25088	-	4096	Fully Connected Layer 1
FC2	4096	-	4096	Fully Connected Layer 2
FC3 (Softmax)	4096	-	1000	최종 클래스 예측

1. ILSVRC 2014 대회 성과: 분류(Classification) 부문 2위, 국소화(Localization) 부문 1위
 - Top-5 Accuracy 92.7%
2. VGG19가 VGG16보다 약간 더 성능이 좋지만, 더 많은 메모리를 사용함
3. 데이터 증강
 - 이미지 크롭(Random Cropping), 수평 반전(Horizontal Flipping), RGB 채널 강도 조절(Color Jittering) 적용
4. 학습 파라미터
 - Dropout 0.5(FC 1, 2층에서 사용)
 - Batch size 256, Optimizer : SGD + Momentum(0.9)
 - 초기 학습률 0.01, 검증 정확도가 정체될 때마다 1/10 감소 (2회)
 - L2 가중치 감소(weight decay) $5e-4$

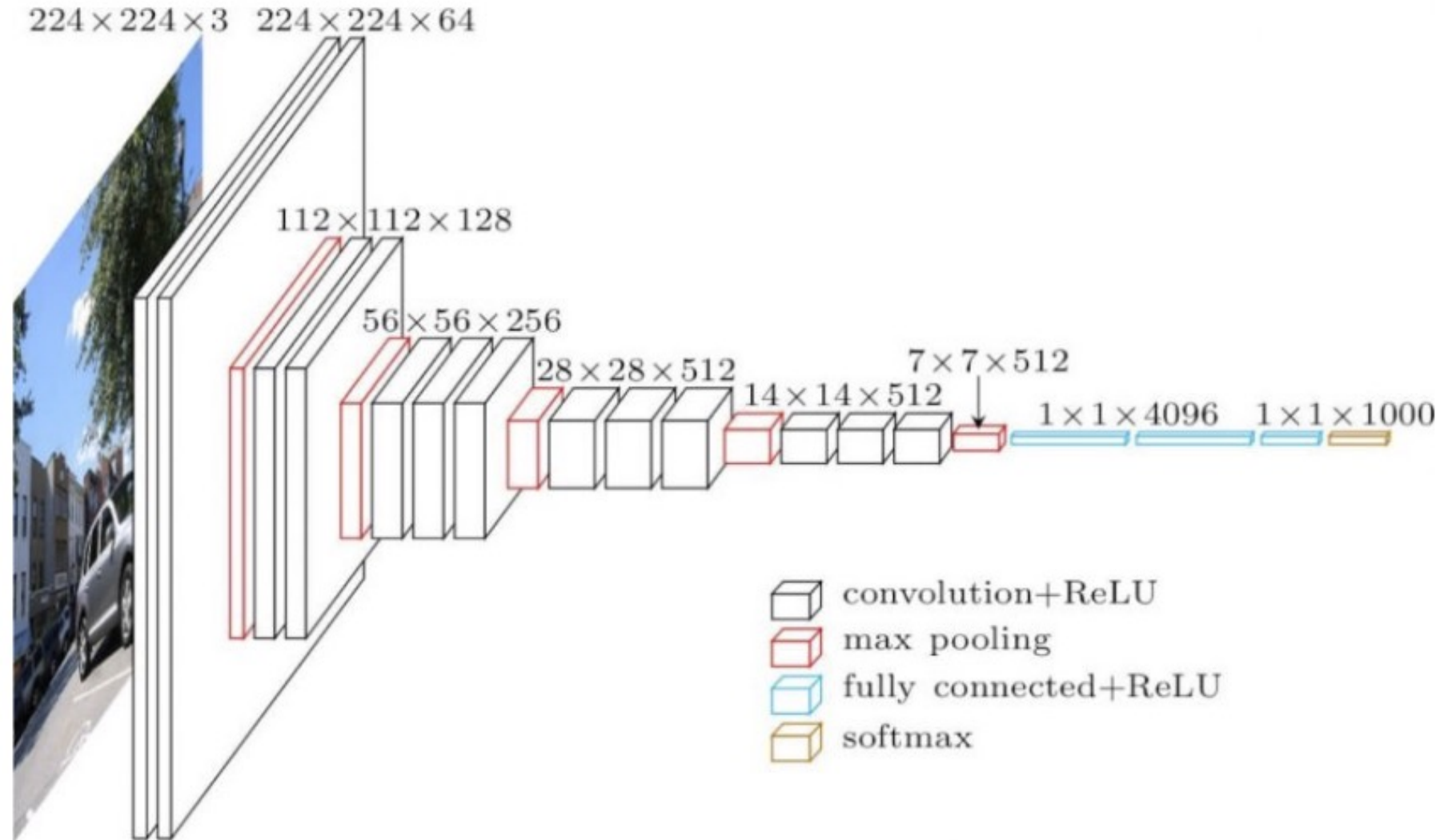
1. 왜 마지막 FCN을 통과할까요?
2. 왜 모든 Conv 레이어에 Max-Pooling을 통과시키지 않을까요?
3. 깊은 네트워크를 쌓는 것의 장점과 단점은 무엇일까요?

장점

1. 단순성과 일관성:
동일한 3×3 필터와 2×2 최대 풀링 구조를 반복 사용함으로써, 네트워크 설계가 매우 단순해지고 이해하기 쉬워집니다.
2. 효과적인 특징 학습:
깊은 네트워크 구조와 누적된 비선형성 덕분에, 보다 복잡하고 추상적인 특징들을 효과적으로 학습할 수 있습니다.
3. 전이 학습(Transfer Learning):
VGGNet은 사전 학습된 모델로 널리 사용되며, 다양한 컴퓨터 비전 과제에서 전이 학습의 베이스 모델로 활용됩니다.

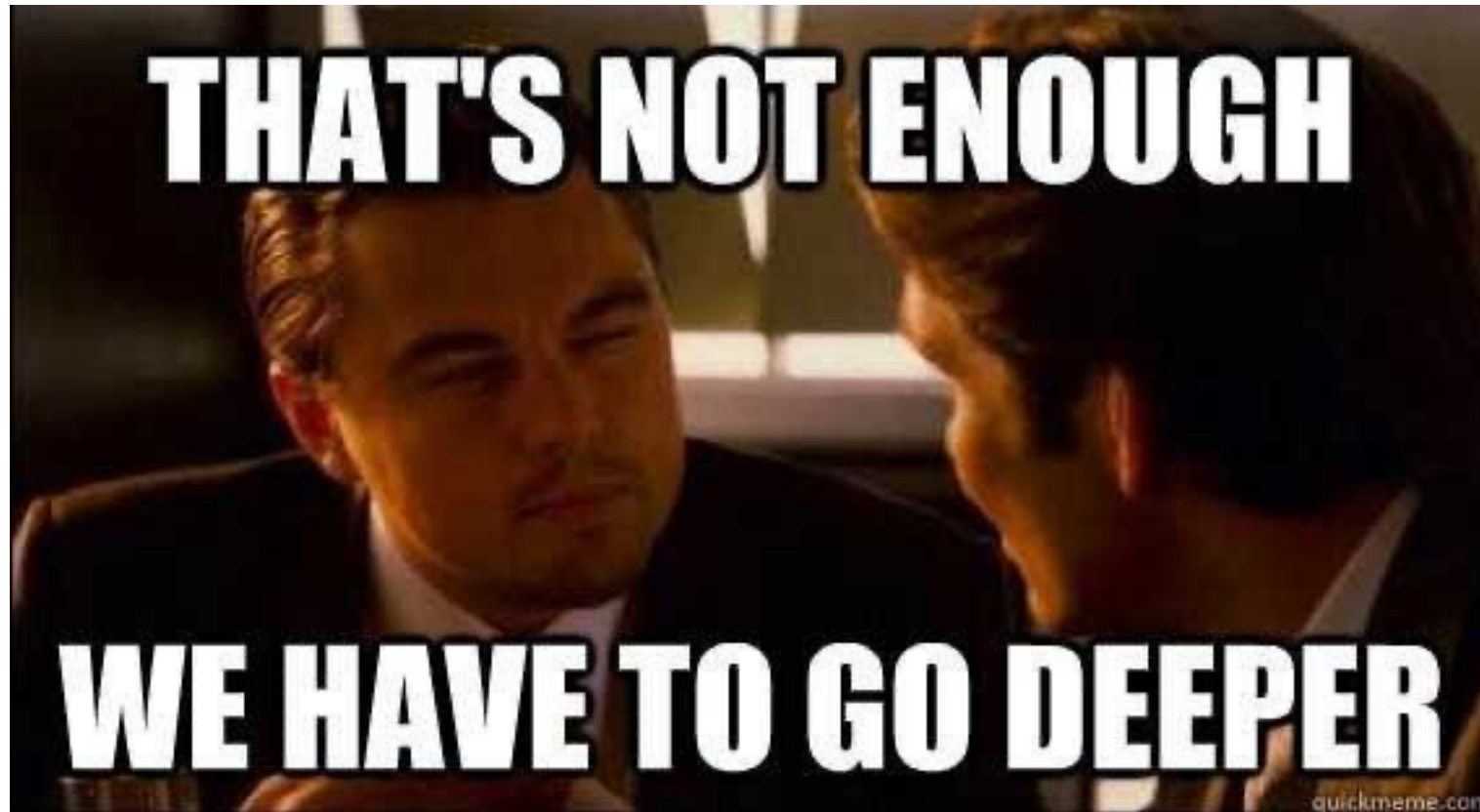
고려사항

1. 계산 및 메모리 비용:
깊은 구조와 많은 파라미터 수 때문에 학습 및 추론 시 계산 자원과 메모리 사용량이 많습니다.
2. 과적합 문제:
충분한 데이터와 적절한 정규화 기법(예: 드롭아웃, 데이터 증강 등)을 사용하지 않으면 과적합의 위험이 있습니다.



VGG16 Architecture

Inception Net (2014.09)



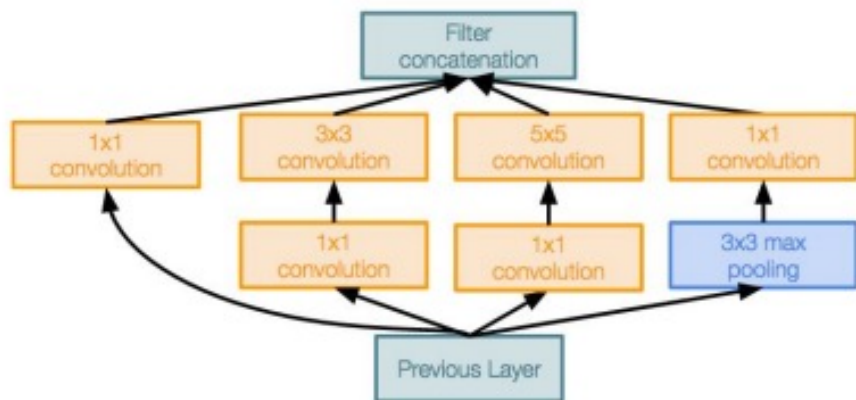
GoogLeNet

- GoogLeNet은 2014년 ImageNet 대회(ILSVRC)에서 1위를 차지한 모델로, Google에서 개발했습니다.
- 네트워크 설계의 효율성과 성능을 동시에 고려하며, Inception Module이라는 새로운 설계 방식을 도입했습니다.
- 주요 기여
 - 네트워크 깊이와 폭의 조화: 네트워크를 깊게 설계하면서도 연산 효율성을 유지.
 - Inception Module 도입: 다양한 필터 크기를 병렬로 적용하여 정보를 다각도로 학습.
 - 경량 모델: 기존 딥러닝 모델보다 적은 파라미터로 높은 성능을 달성.
- "Going deeper with convolutions"
 - <https://arxiv.org/pdf/1409.4842>

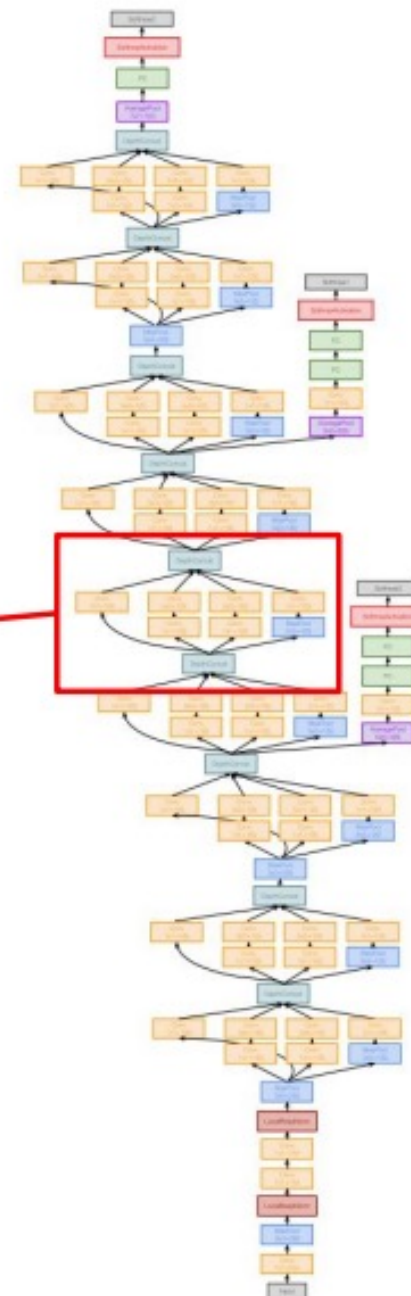
GoogLeNet

Deeper networks, with computational efficiency

- 22층
- Inception 모듈 사용
- No FC Layers
- 약 6.8M개의 파라미터
- 14년도 이미지넷 우승



Inception module



Inception Net의 설계 동기

1. 깊은 네트워크의 장점과 계산 효율 문제

깊은 네트워크의 장점: 네트워크 깊이를 늘리면 더 복잡하고 추상적인 특징 표현을 학습할 수 있어 이미지 분류, 검출 등 다양한 컴퓨터 비전 문제에서 성능 향상을 기대할 수 있습니다.

2. 계산 비용 문제: 하지만 깊은 네트워크는 계산량과 파라미터 수가 급격히 증가하는 문제가 발생합니다.

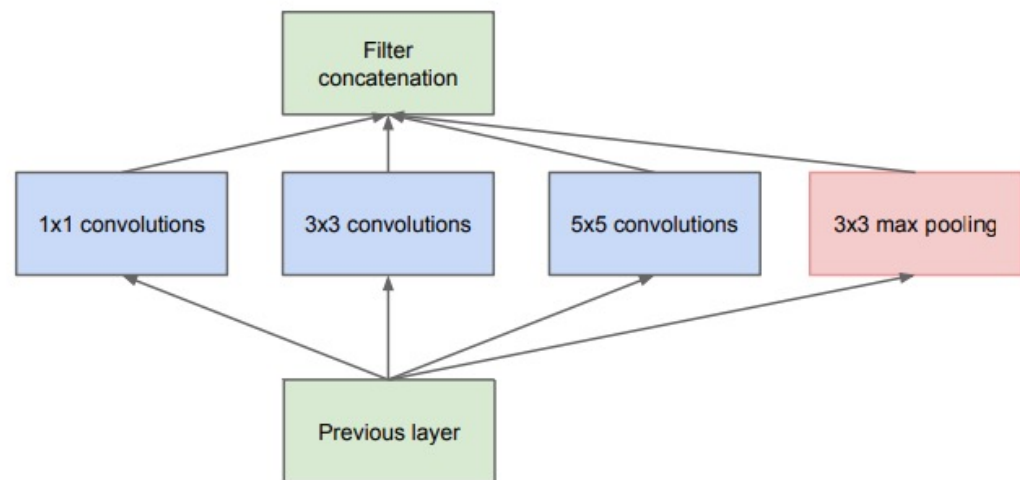
예를 들어, 단순히 레이어 수를 늘리는 경우 메모리 사용량 및 연산량이 기하급수적으로 늘어나게 됩니다.

Inception 구조

InceptionNet은 네트워크 내에서 여러 스케일의 특징을 동시에 추출하면서도 계산 효율을 극대화하기 위해

"Inception module"이라는 새로운 블록 구조를 도입했습니다.

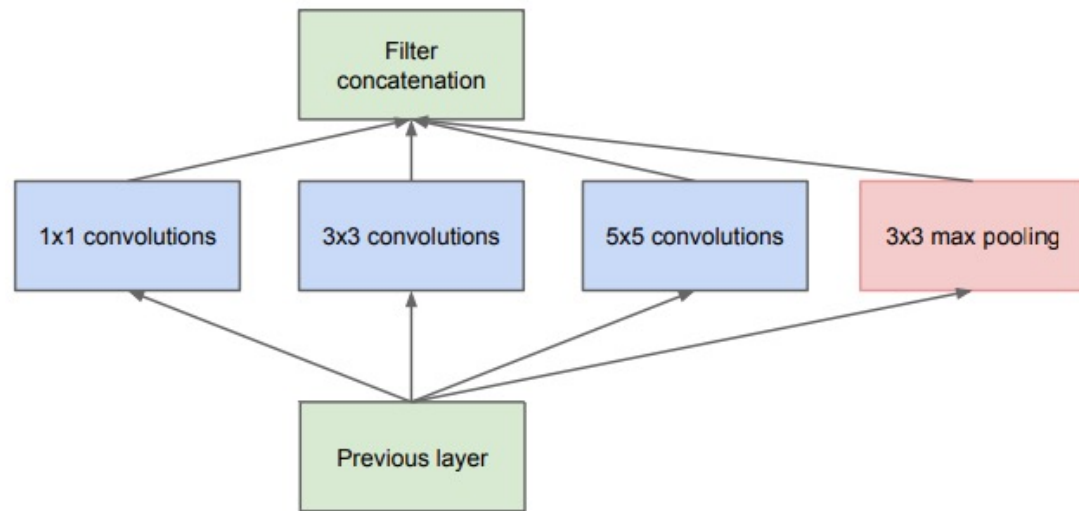
- 핵심 아이디어: 하나의 모듈 내에서 **1×1, 3×3, 5×5 등의 다양한 크기의 컨볼루션 필터**와, 최대 풀링 연산을 병렬로 적용한 후 그 결과를 **채널 방향**으로 합치는 방식입니다.
- 목표: 각기 다른 크기의 필터를 사용하면, 이미지 내에서 작은 패턴부터 큰 패턴까지 다양한 공간적 정보를 동시에 캡처할 수 있으며, 이를 통해 네트워크가 보다 풍부한 표현을 학습할 수 있습니다.



(a) Inception module, naïve version

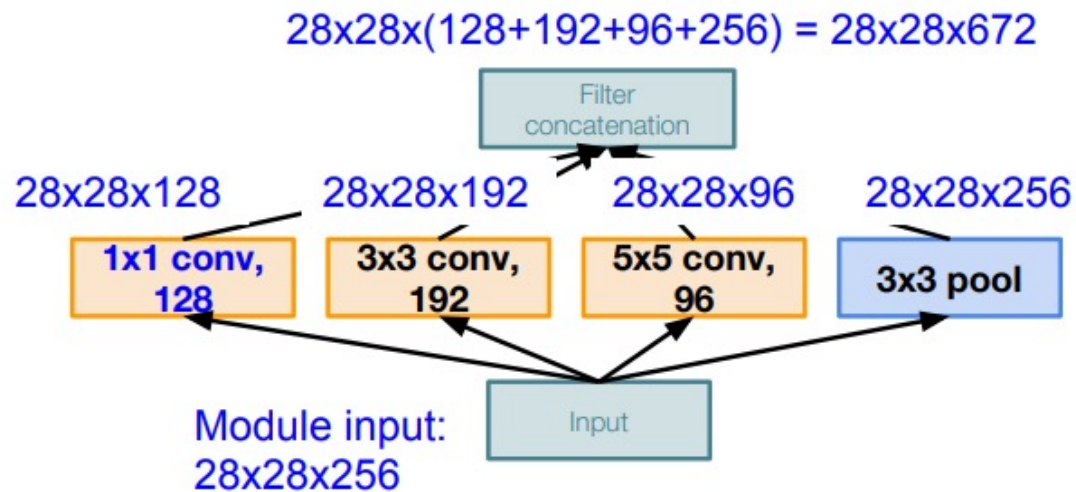
Inception 구조

- 장점
모든 계층에서 다중 해상도(Multi-resolution) 특징을 추출 가능
→ 더 높은 유연성과 향상된 성능 제공
- 문제점
연산 복잡도(Computational complexity)
- 연산량과 출력크기를 계산해보세요.
(stride = 1, SAME padding)
 - 1x1 Conv (128 filters)
 - 3x3 Conv (192 filters)
 - 5x5 Conv (96 filters)



(a) Inception module, naïve version

Inception 구조

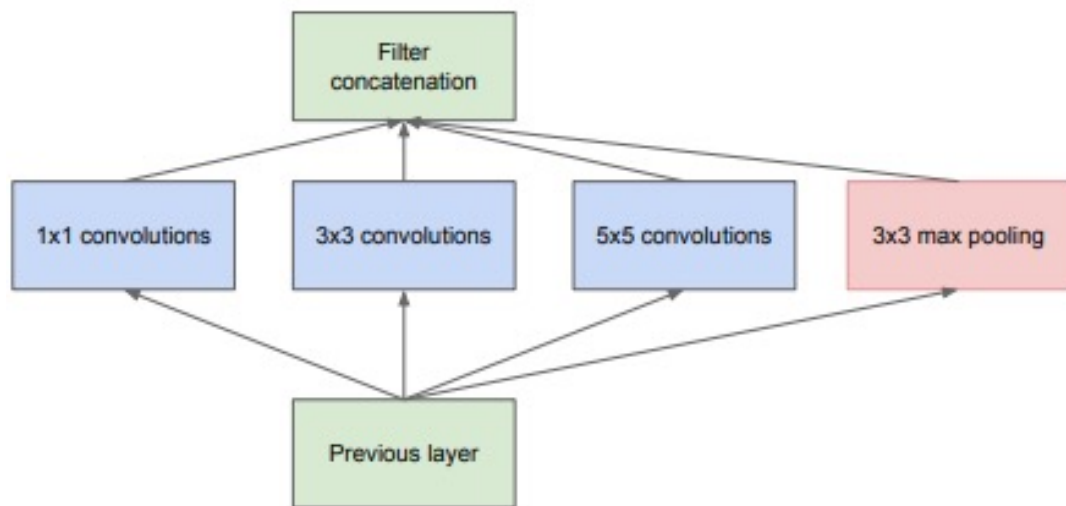


Naive Inception module

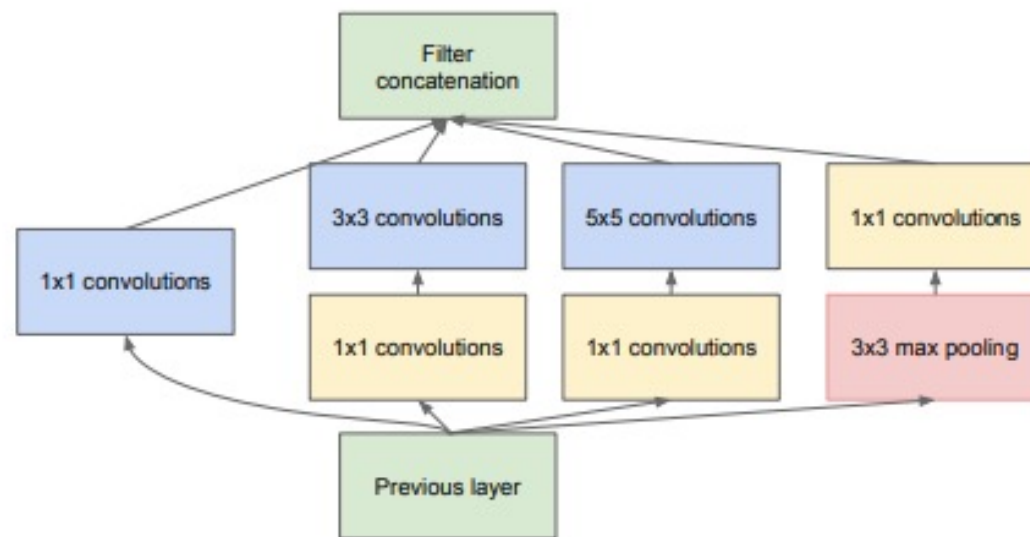
연산량 분석 (Number of Operations)

연산 유형	공식	연산량(ops)
1x1 합성곱 (128 filters)	$(1 \times 1 \times 256) \times 128 \times (28 \times 28)$	102M
3x3 합성곱 (192 filters)	$(3 \times 3 \times 256) \times 192 \times (28 \times 28)$	447M
5x5 합성곱 (96 filters)	$(5 \times 5 \times 256) \times 96 \times (28 \times 28)$	230M
3x3 Max Pooling	$(3 \times 3) \times 256 \times (28 \times 28)$	75M
총 연산량 (Total)		854M ops

Inception 구조



(a) Inception module, naïve version



(b) Inception module with dimension reductions

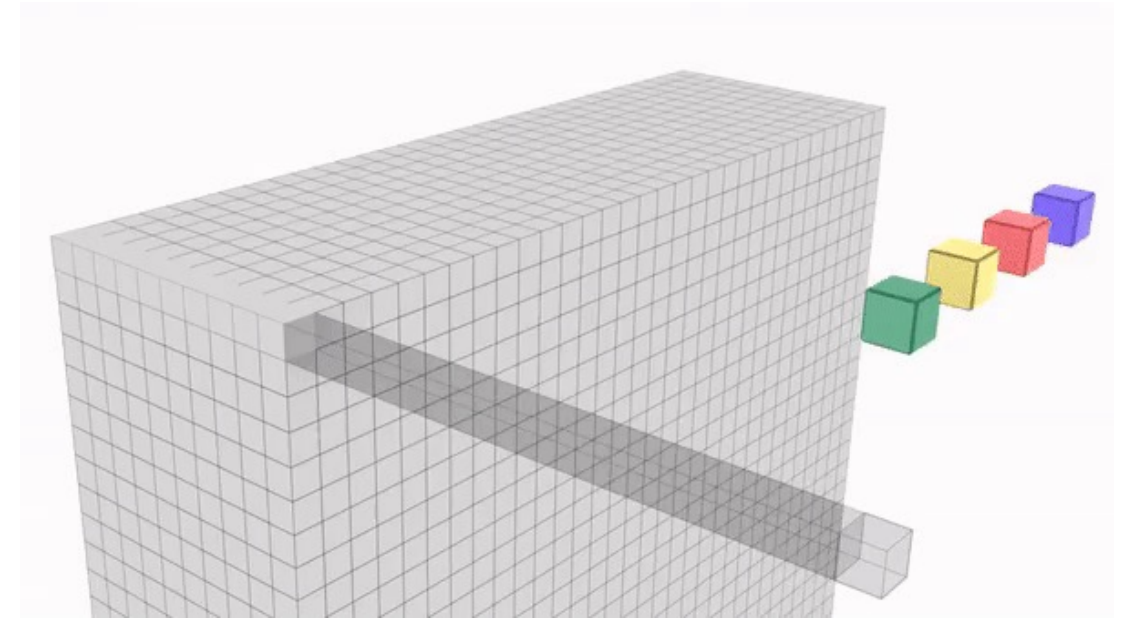
Figure 2: Inception module

1x1 Convolution

- 1x1 컨볼루션은 각 위치에서 1x1 크기의 필터를 입력 이미지 (또는 이전 층의 특징 맵)의 모든 채널에 대해 적용합니다. 즉, 각 픽셀 위치에서 입력 채널들의 값을 **선형 결합**하여 새로운 값을 생성합니다.

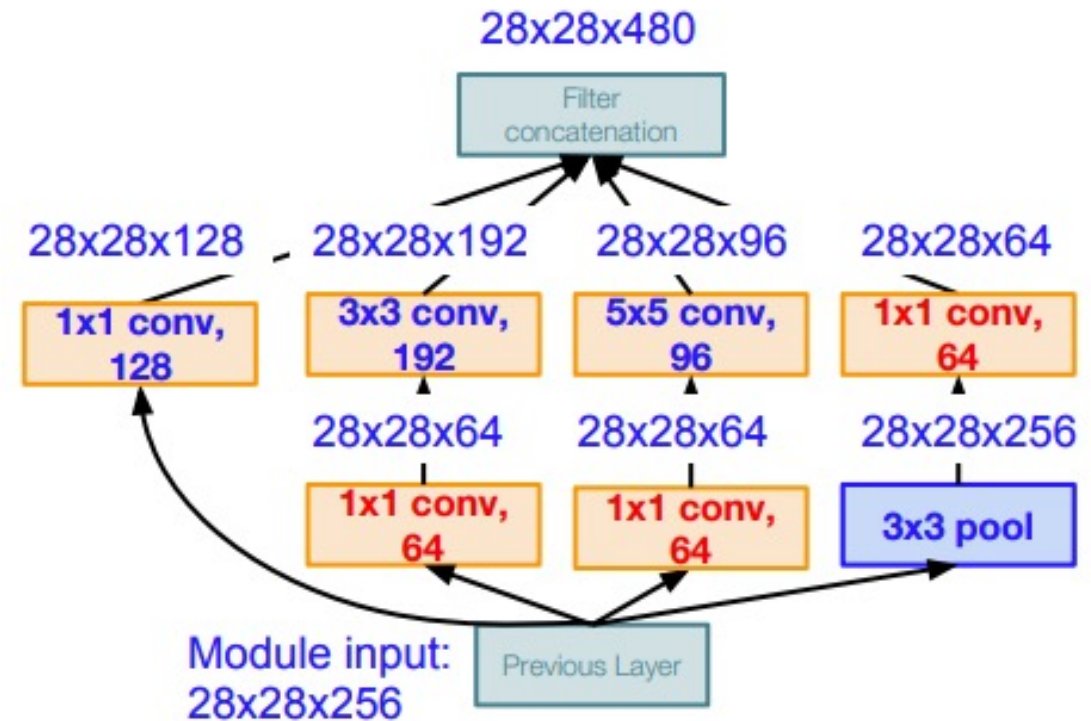
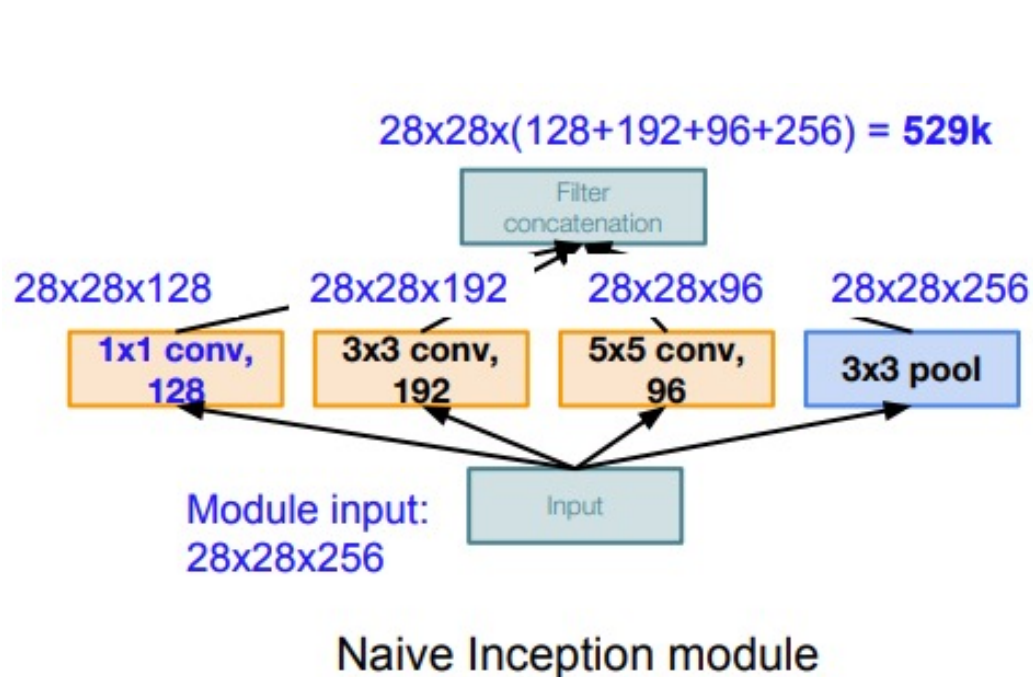
주요 역할

- 차원 축소 : 입력의 채널 수(깊이)를 줄이는 데 효과적
- 채널 간 정보 결합 : 각 픽셀에서 여러 채널의 정보를 선형 결합하여 새로운 특징을 만들어내므로, 채널 간의 상호작용을 학습 가능
- 비선형성 도입 : 일반적으로 ReLU와 같은 활성화 함수를 뒤 따르므로, 단순한 선형 결합 이상의 비선형 표현을 학습



Inception 구조

- 동일한 병렬 계층 구조를 사용하되, "1x1 합성곱, 64개 필터" 병목 계층(bottleneck layer)을 추가



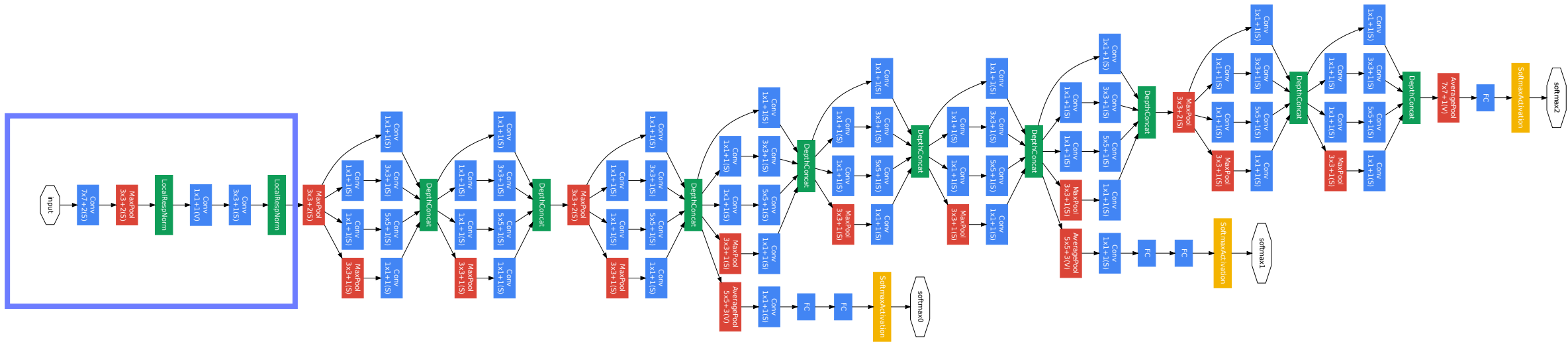
Inception 구조

- 동일한 병렬 계층 구조를 사용하되, "1×1 합성곱, 64개 필터" 병목 계층(bottleneck layer)을 추가

연산량(Conv Ops):

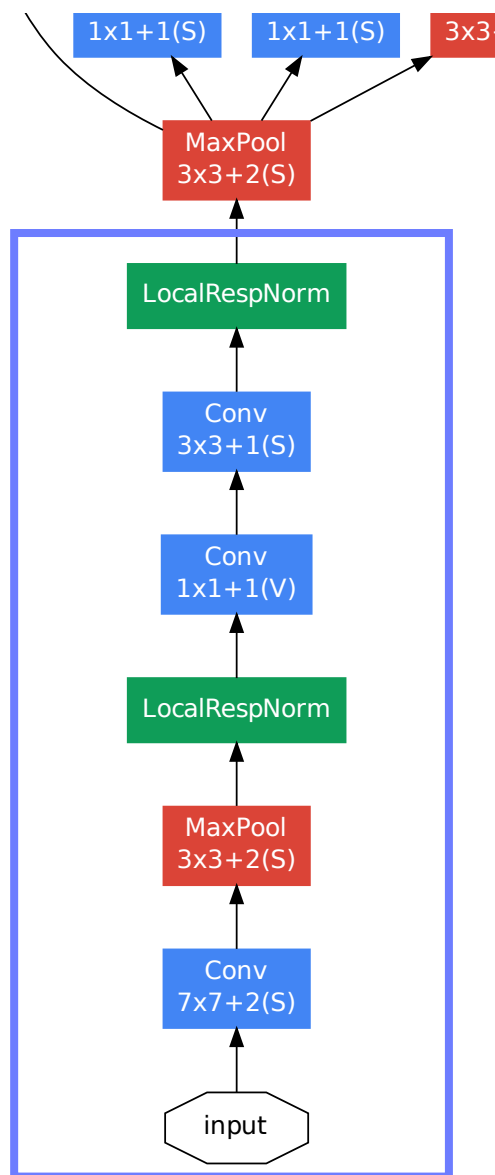
연산 유형	연산 공식
1×1 합성곱 (64 필터)	$28 \times 28 \times 64 \times 1 \times 1 \times 256$
1×1 합성곱 (64 필터)	$28 \times 28 \times 64 \times 1 \times 1 \times 256$
1×1 합성곱 (128 필터)	$28 \times 28 \times 128 \times 1 \times 1 \times 256$
3×3 합성곱 (192 필터)	$28 \times 28 \times 192 \times 3 \times 3 \times 64$
5×5 합성곱 (96 필터)	$28 \times 28 \times 96 \times 5 \times 5 \times 64$
1×1 합성곱 (64 필터)	$28 \times 28 \times 64 \times 1 \times 1 \times 256$

 총 연산량(Total): 358M ops

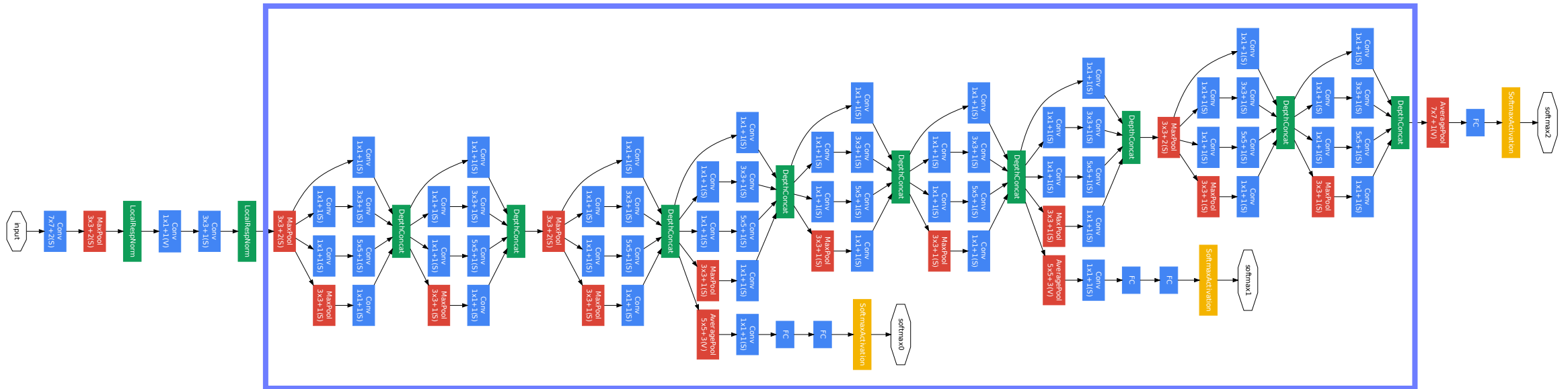


Stem Network

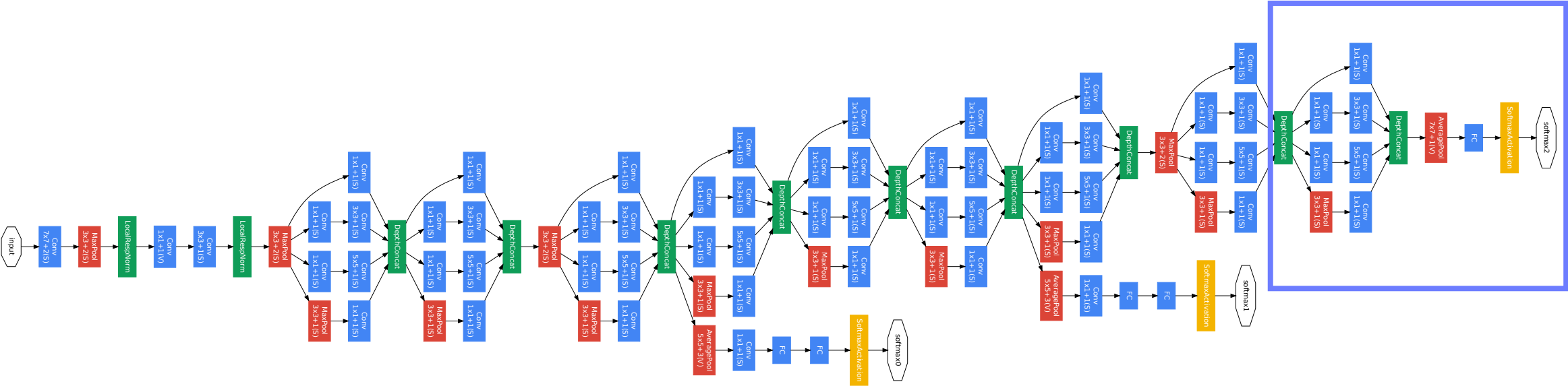
- Conv – Pool – Conv – Conv – Pool



Stacked Inception Modules

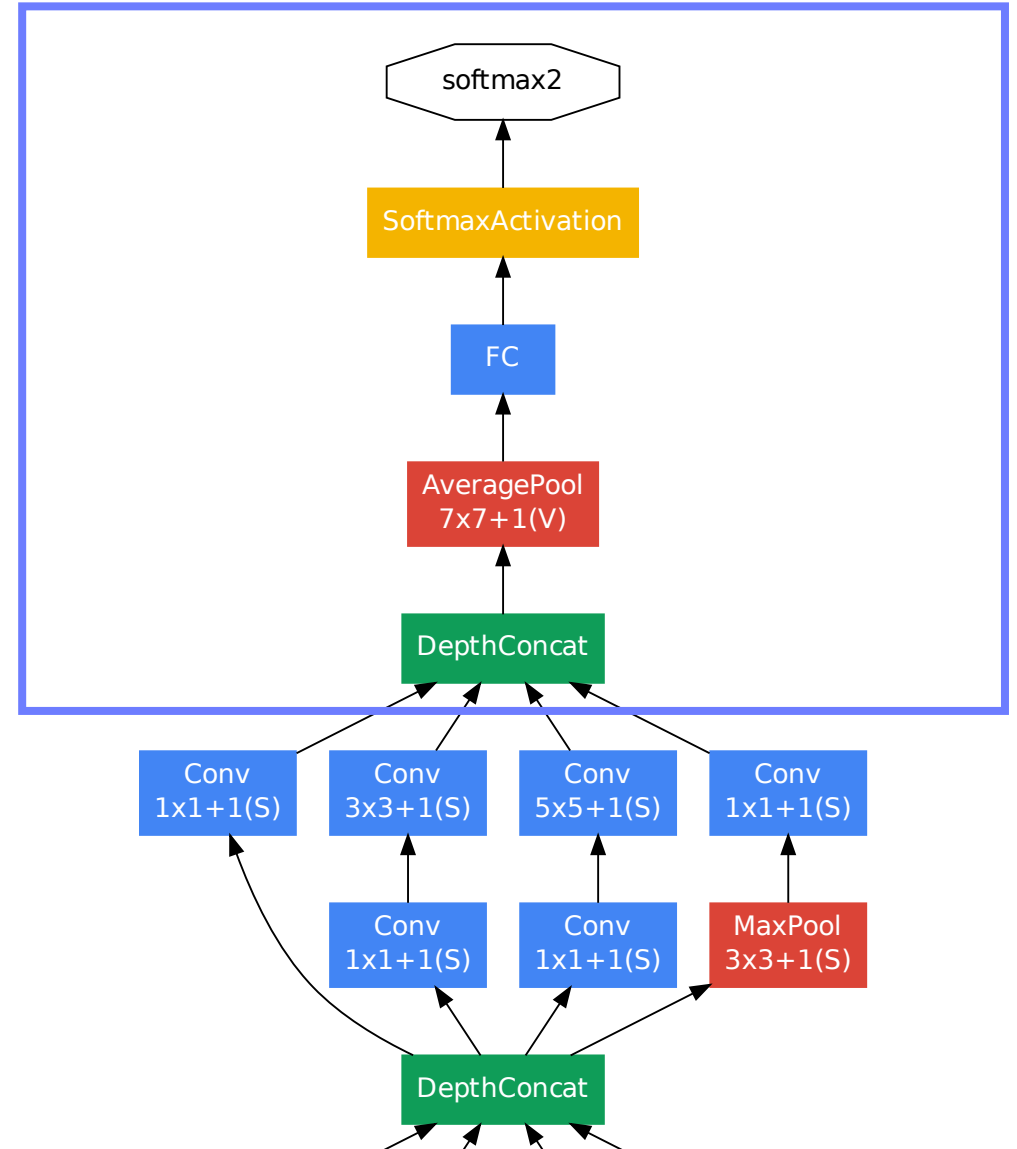


Classifier



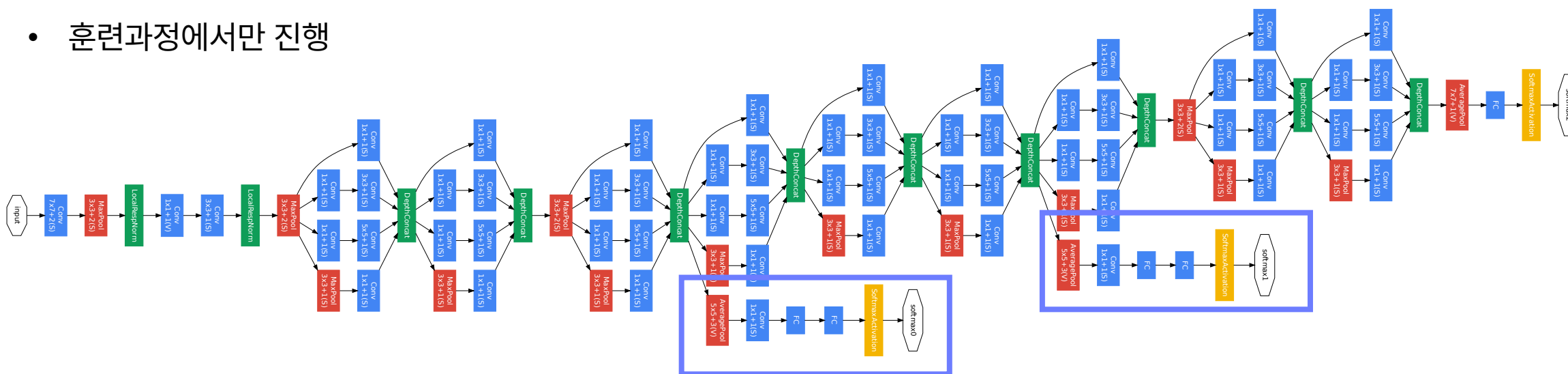
Classifier

- 마지막 Inception 모듈 이후, Global Average Pooling Layer을 사용하여 각 특징 맵을 공간적으로 평균화(spatially average)한 후, 최종 완전 연결 계층(FC Layer)으로 전달됨.
- 더 이상 여러 개의 비용이 많이 드는 FC 계층을 사용하지 않음 (단, 마지막 출력에서 각 클래스로 매핑하는 FC 계층은 예외).
- FC 계층을 제거함으로써, 전체 네트워크 아키텍처가 더 희소한 연결(Sparser Connections)을 형성하게 되며, 결과적으로 연산이 더 효율적으로 수행됨.



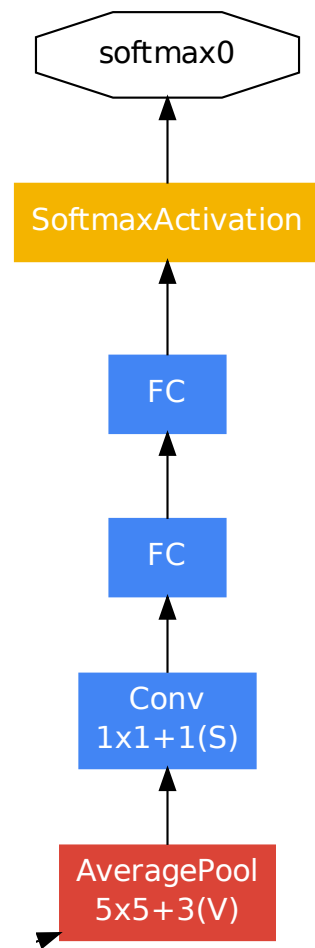
Auxiliary Classifier

- 중간 계층에 보조 분류기를 추가하여, 깊은 네트워크에서 발생할 수 있는 기울기 소실 문제를 완화하고, 학습을 안정화시킵니다.
- $\text{Loss} = \text{Outloss} + 0.3 * (\text{aux1_loss} + \text{aux2_loss})$
- 훈련과정에서만 진행



Auxiliary Classifier

- 중간 계층에 보조 분류기를 추가하여, 깊은 네트워크에서 발생할 수 있는 기울기 소실 문제를 완화하고, 학습을 안정화시킵니다.
- $\text{Loss} = \text{Outloss} + 0.3 * (\text{aux1_loss} + \text{aux2_loss})$
- 훈련과정에서만 진행



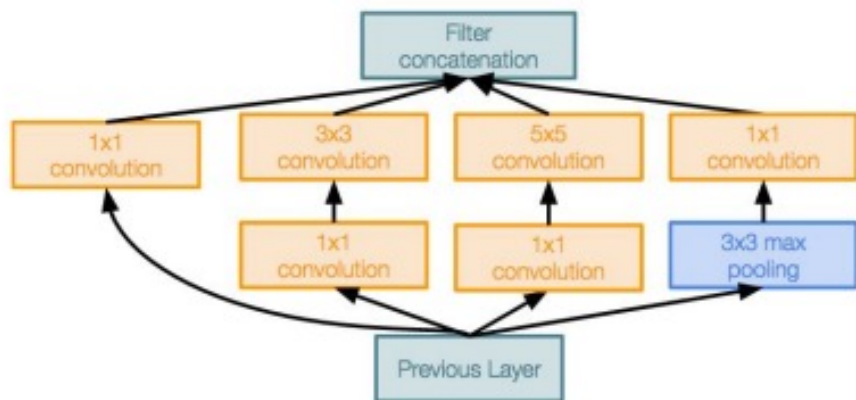
	type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
	convolution	7×7/2	112×112×64	1							2.7K	34M
	max pool	3×3/2	56×56×64	0								
	convolution	3×3/1	56×56×192	2		64	192				112K	360M
	max pool	3×3/2	28×28×192	0								
	inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
	inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
	max pool	3×3/2	14×14×480	0								
	inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
aux1 ←	inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
	inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
	inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
aux2 ←	inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
	max pool	3×3/2	7×7×832	0								
	inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
	inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
GAP	avg pool	7×7/1	1×1×1024	0								
	dropout (40%)		1×1×1024	0								
	linear		1×1×1000	1							1000K	1M
	softmax		1×1×1000	0								

Table 1: GoogLeNet incarnation of the Inception architecture

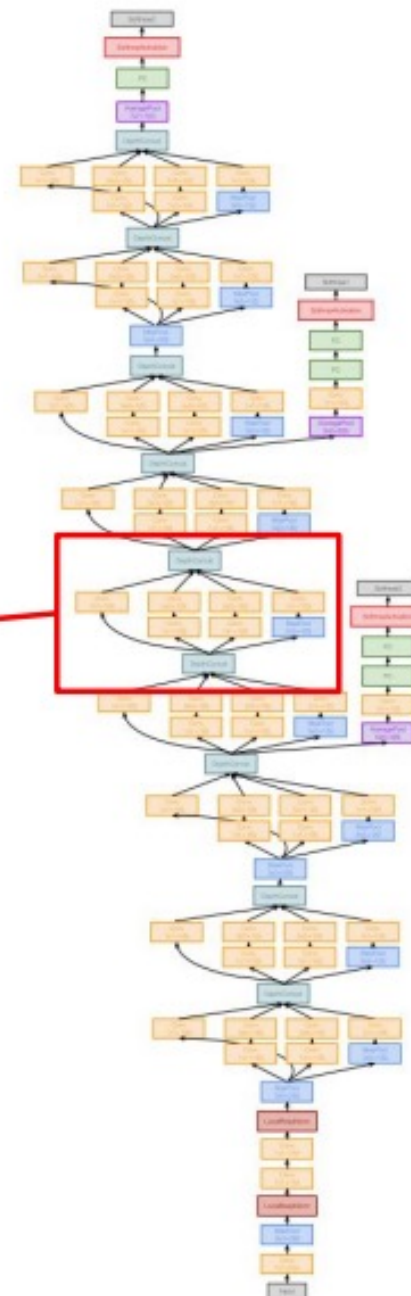
GoogLeNet

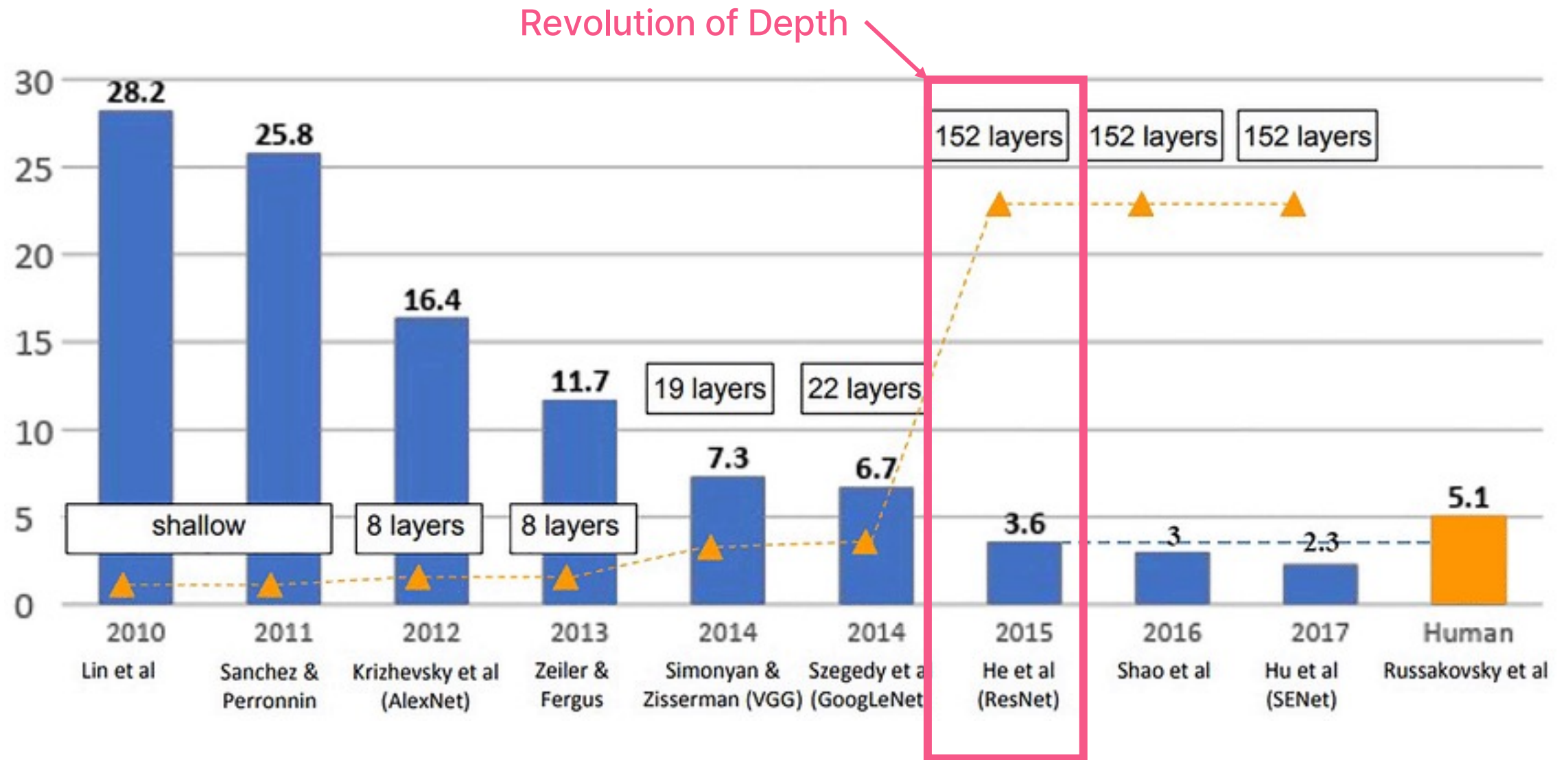
Deeper networks, with computational efficiency

- 22층
- Inception 모듈 사용
- No FC Layers
- 6.8M개의 파라미터 (알렉스넷의 1/12배)
- 14년도 이미지넷 우승
(6.7% top 5 error)



Inception module





ResNet

- ResNet(Residual Network)은 2015년 ILSVRC에서 1위를 차지한 모델로, Microsoft Research에서 개발했습니다.
- ResNet은 152개의 계층을 성공적으로 학습하며 딥러닝 모델의 깊이에 관한 새로운 기준을 제시했습니다.
- 주요 기여
 - ResNet은 Gradient Vanishing 문제를 해결하며 매우 깊은 네트워크 설계를 가능하게 했습니다.
 - Residual Connection을 도입하여 네트워크 학습을 안정화하고, 성능 향상과 일반화에 기여했습니다.
 - "사람보다 더 정확한" 모델로, 이미지 분류의 성능이 인간을 넘어서는 계기를 마련했습니다.
 - <https://arxiv.org/pdf/1512.03385>

**Is learning better networks
as easy as stacking more layers?**

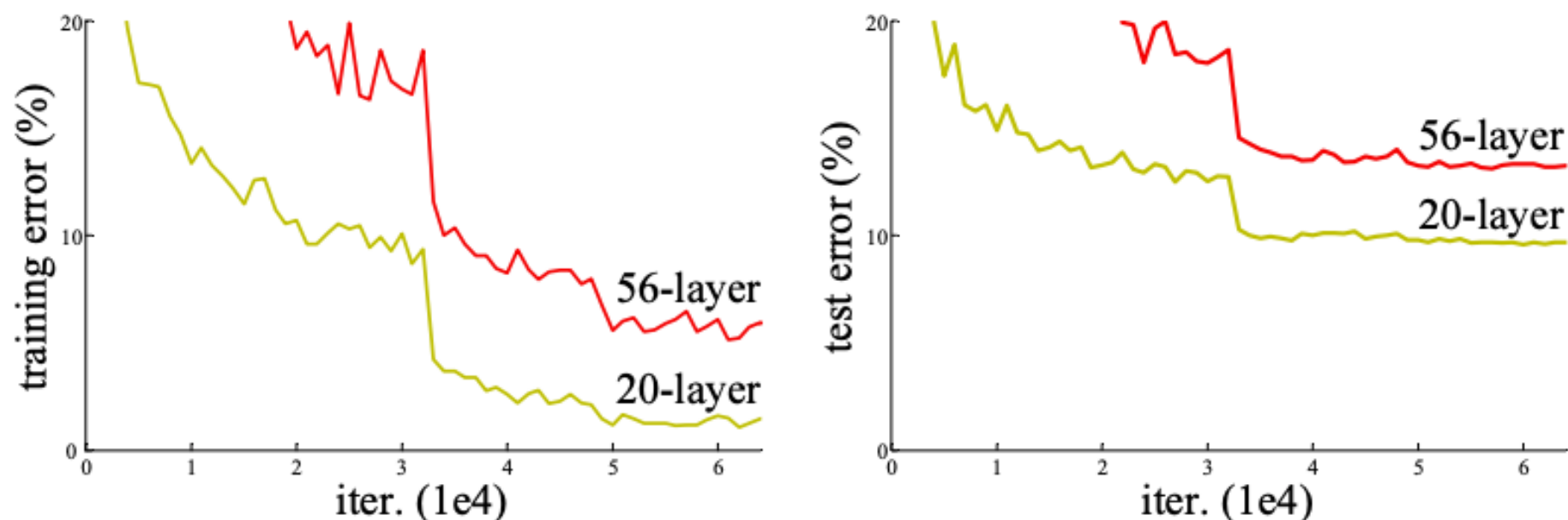


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Residual Learning (잔차학습)

- 여러 개의 쌓인 레이어가 근사해야 하는 기저 매핑을 $H(x)$ 라고 합시다.
- 여기서 x 는 해당 레이어들의 첫 입력입니다.
- 여러 비선형 레이어가 복잡한 함수를 근사할 수 있다면,
동일한 네트워크가 잔차 함수 $F(x)=H(x)-x$ 도 근사할 수 있습니다.
- 즉, 원래의 함수 $H(x)$ 대신 $F(x)+x$ 를 학습하도록 재구성하는 것입니다.

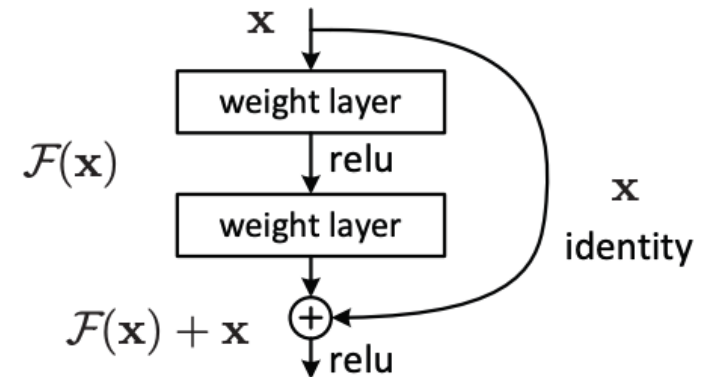
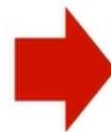
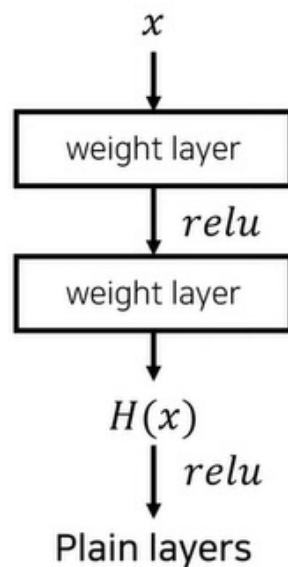


Figure 2. Residual learning: a building block.

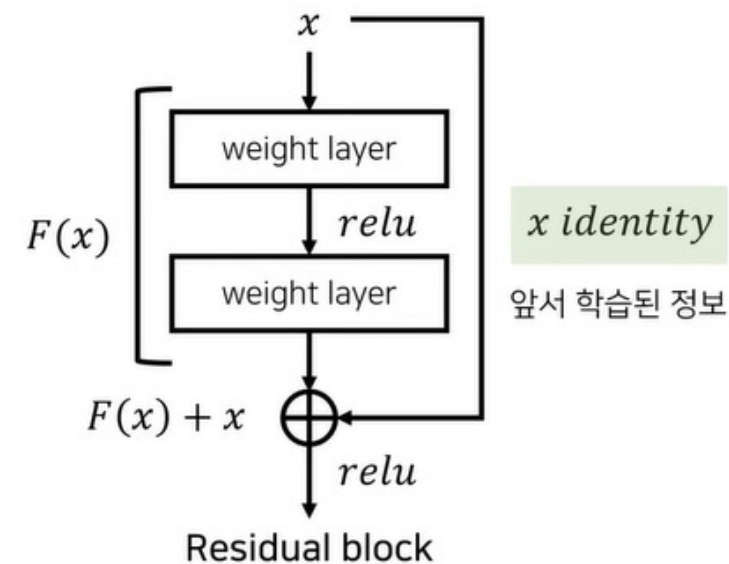
Shortcut connection(Skip connection)

- 이전 모델 : $y = H(x)$
- ResNet : $y = F(x) + x$
- 즉, 잔차 $F(x) = H(x) - x$ 를 모델링



학습이 잘 되는 형태로 변경

WHY?



왜 학습이 더 잘 될까?

- 왜 잔차 함수 $F(x)=H(x)-x$ 를 직접 학습하는 것이 원래 함수 $H(x)$ 를 학습하는 것보다 최적화가 쉬울까?

만약 최적의 함수 $H(x)$ 가 항등매핑, 즉 $H(x) = x$ 라면, $F(x) = 0$ 이 되어야 한다.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad 0_{m \times n} = \begin{pmatrix} 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}_{m \times n}$$

왜 학습이 더 잘 될까?

- 왜 잔차 함수 $F(x)=H(x)-x$ 를 직접 학습하는 것이 원래 함수 $H(x)$ 를 학습하는 것보다 최적화가 쉬울까?

작은 변화(Residual)만 학습

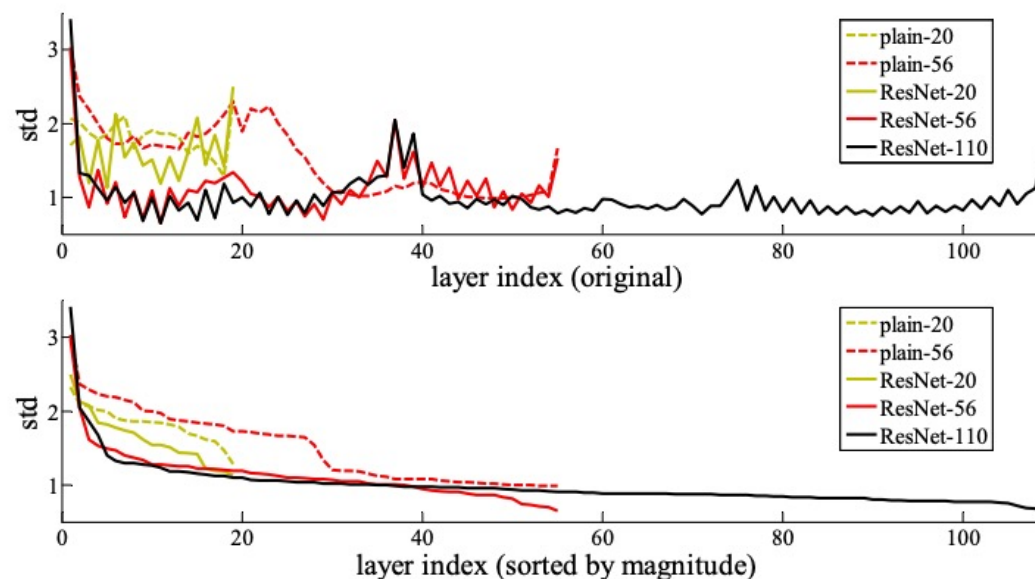


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top:** the layers are shown in their original order. **Bottom:** the responses are ranked in descending order.

왜 학습이 더 잘 될까?

- 왜 잔차 함수 $F(x)=H(x)-x$ 를 직접 학습하는 것이 원래 함수 $H(x)$ 를 학습하는 것보다 최적화가 쉬울까?

Gradient 흐름 개선

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \left(\frac{\partial F(x)}{\partial x} + I \right)$$

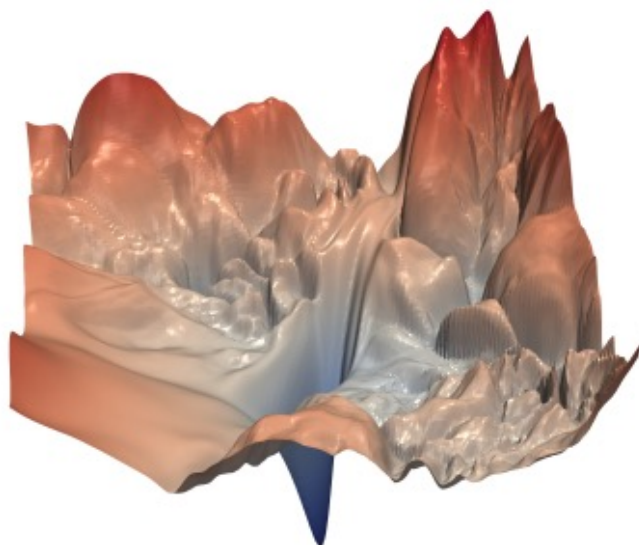
$\frac{\partial F(x)}{\partial x}$ 가 작아서 gradient가 소실되더라도, 항등 행렬 I 덕분에

$$\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial y} \quad \text{가 되어 gradient가 직접 } x \text{로 전달됩니다.}$$

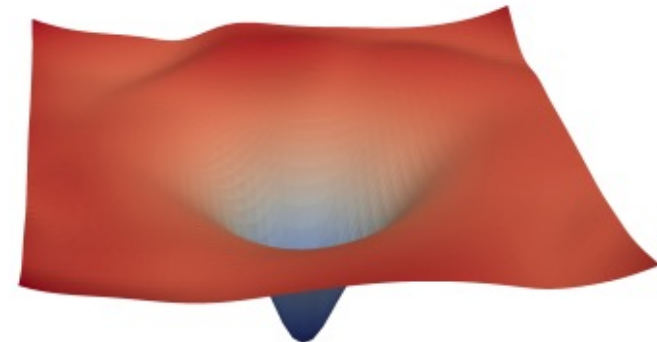
왜 학습이 더 잘 될까?

- 왜 잔차 함수 $F(x)=H(x)-x$ 를 직접 학습하는 것이 원래 함수 $H(x)$ 를 학습하는 것보다 최적화가 쉬울까?

Smoother Loss Landscape



(a) without skip connections



(b) with skip connections

Figure 1: The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

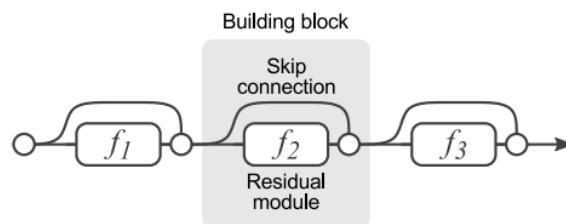
Visualizing the Loss Landscape of Neural Nets

Hao Li¹, Zheng Xu¹, Gavin Taylor², Christoph Studer³, Tom Goldstein¹
¹University of Maryland, College Park ²United States Naval Academy ³Cornell University
{haoli, xuzh, tomg}@cs.umd.edu, taylor@usna.edu, studer@cornell.edu

왜 학습이 더 잘 될까?

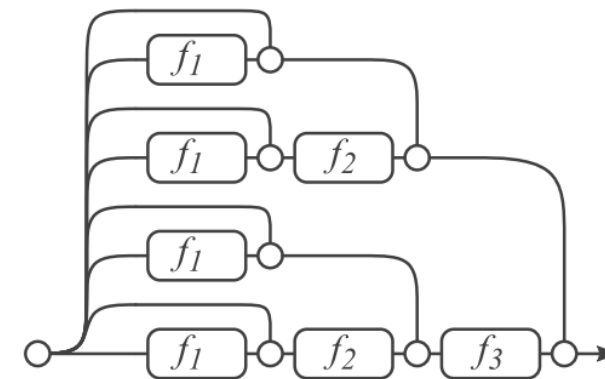
- 왜 잔차 함수 $F(x)=H(x)-x$ 를 직접 학습하는 것이 원래 함수 $H(x)$ 를 학습하는 것보다 최적화가 쉬울까?

Residual Networks Behave
Like Ensembles of Relatively
Shallow Networks



(a) Conventional 3-block residual network

=



(b) Unraveled view of (a)

Residual Networks Behave Like Ensembles of
Relatively Shallow Networks

Andreas Veit Michael Wilber Serge Belongie
Department of Computer Science & Cornell Tech
Cornell University

Figure 1: Residual Networks are conventionally shown as (a), which is a natural representation of Equation (1). When we expand this formulation to Equation (6), we obtain an *unraveled view* of a 3-block residual network (b). Circular nodes represent additions. From this view, it is apparent that residual networks have $O(2^n)$ implicit paths connecting input and output and that adding a block doubles the number of paths.

왜 학습이 더 잘 될까?

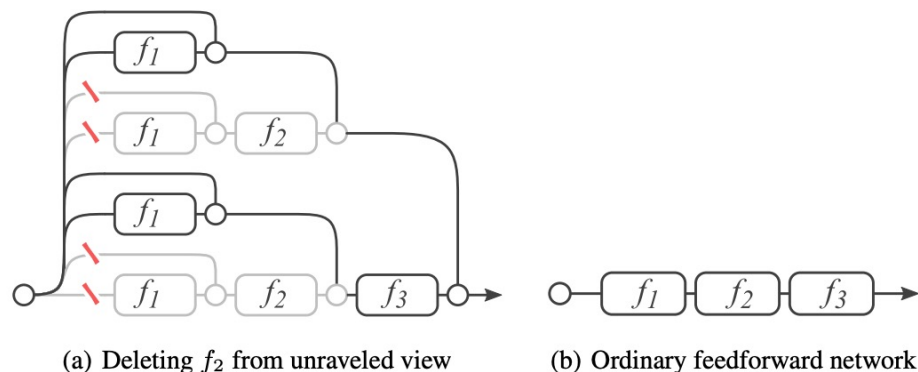


Figure 2: Deleting a layer in residual networks at test time (a) is equivalent to zeroing half of the paths. In ordinary feed-forward networks (b) such as VGG or AlexNet, deleting individual layers alters the only viable path from input to output.

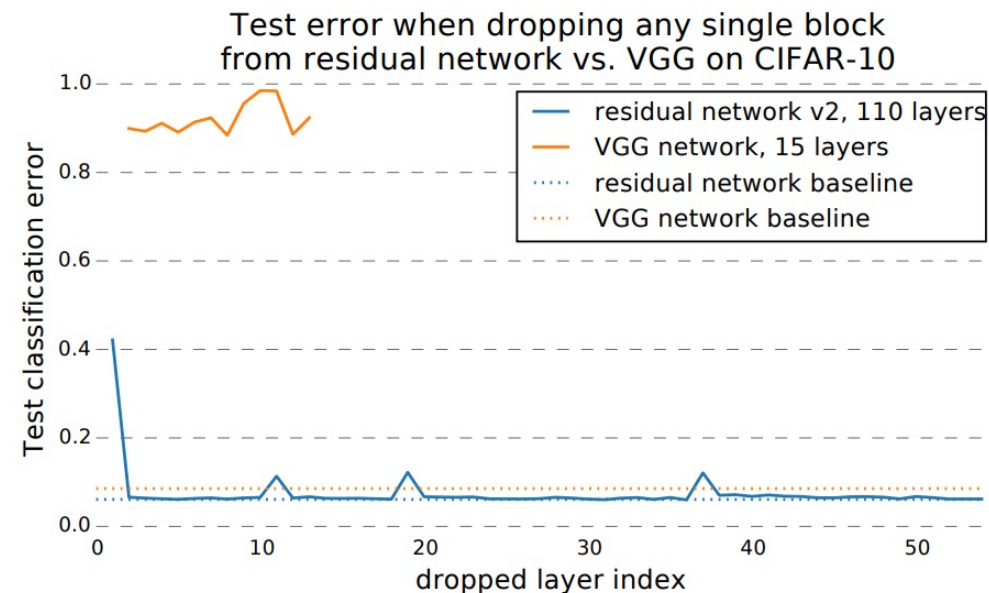
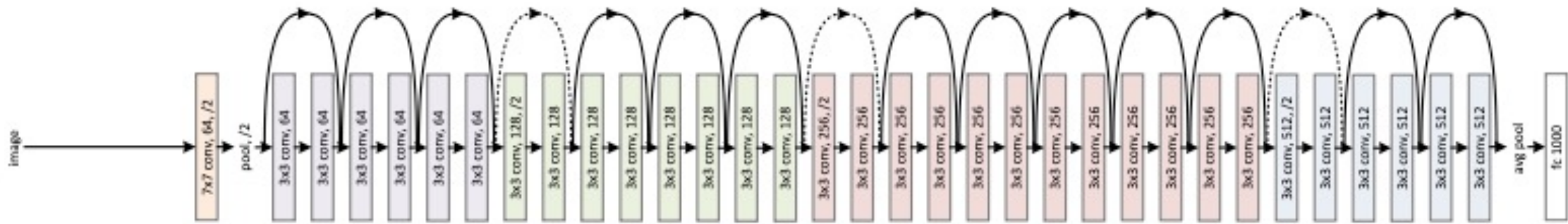
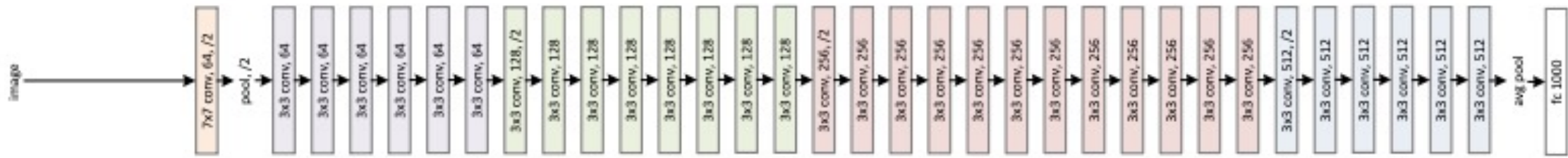


Figure 3: Deleting individual layers from VGG and a residual network on CIFAR-10. VGG performance drops to random chance when any one of its layers is deleted, but deleting individual modules from residual networks has a minimal impact on performance. Removing downsampling modules has a slightly higher impact.

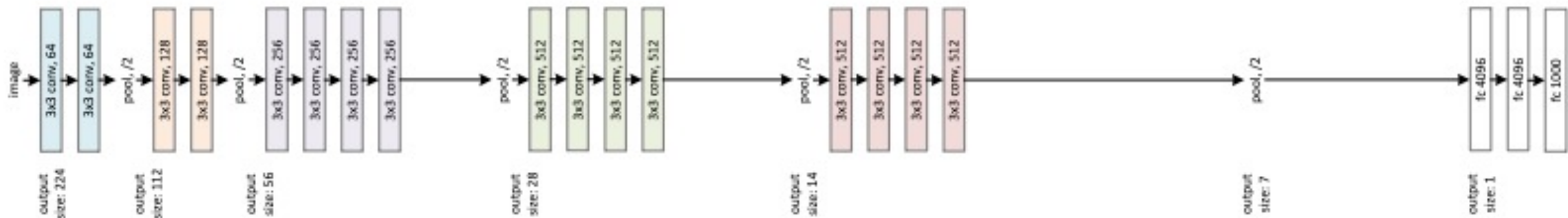
34-layer residual



34-layer plain



VGG-19

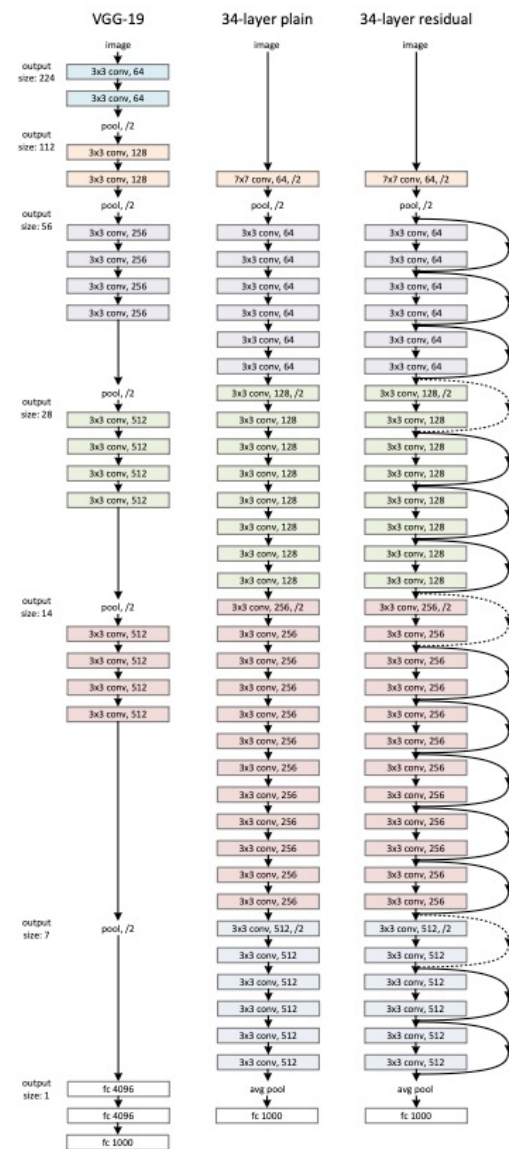


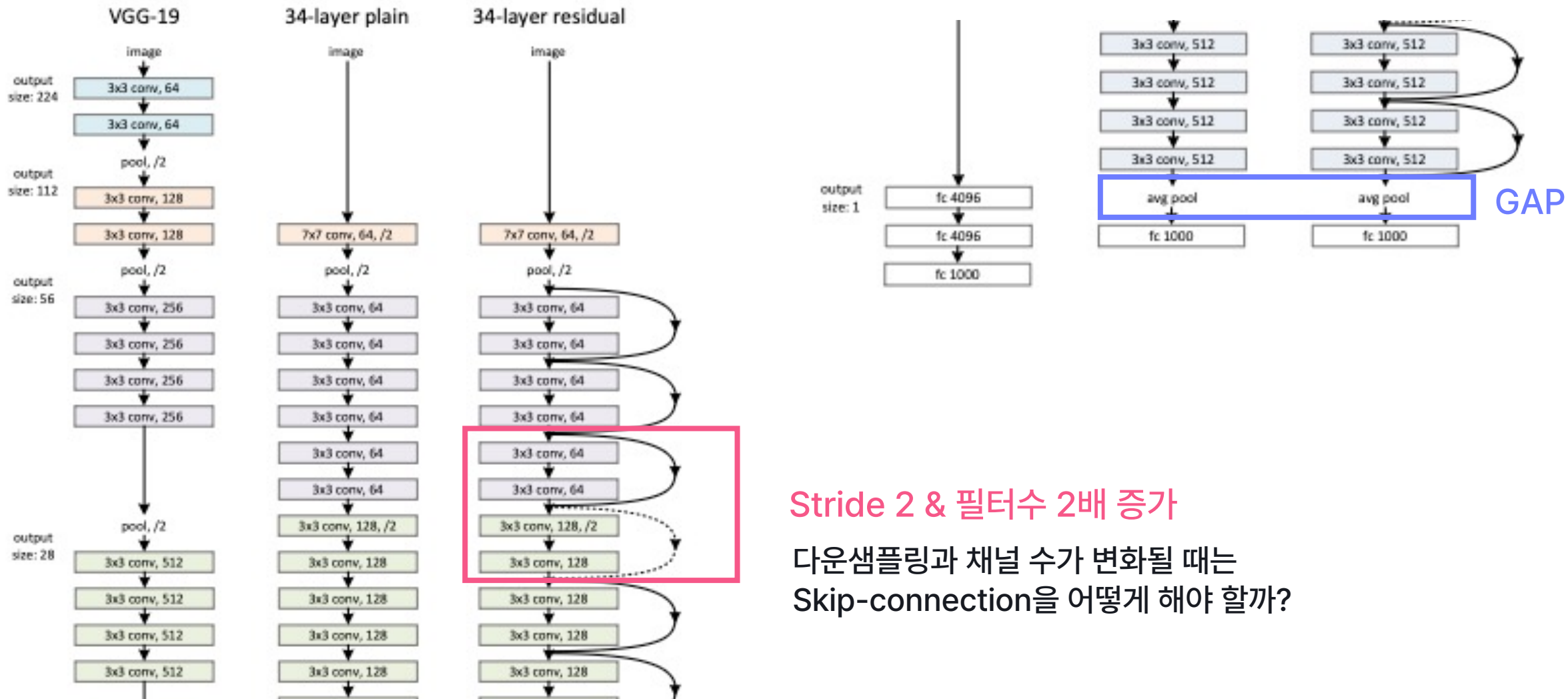
우리의 plain baseline (중앙)은 주로 VGG 네트워크의 철학(좌측)에서 영감을 받았습니다. 컨볼루션 레이어는 대부분 3×3 필터를 사용하며 두 가지 간단한 설계 규칙을 따릅니다.

- 동일한 출력 피쳐 맵 크기의 경우 레이어는 동일한 수의 필터를 가짐
- 피쳐 맵 크기가 절반으로 줄어들면, 각 레이어의 시간 복잡도를 유지하기 위해 필터 수를 두 배로 늘림
- stride 2인 컨볼루션 레이어를 통해 직접 다운샘플링을 수행
- 네트워크는 Global Average Pooling와 softmax를 갖는 1000-클래스 완전 연결 레이어로 마무리
- 그림 3(중앙)에서 가중치가 있는 총 레이어 수는 34개입니다

특히, 우리의 모델은 VGG 네트워크 [41] (그림 3, 좌측)보다 필터 수가 적고 복잡도가 낮다는 점에 주목할 만합니다. 우리의 34층 baseline은 36억 FLOPs(곱셈-덧셈 연산)를 가지며, 이는 VGG-19(196억 FLOPs)의 18%에 불과합니다.

<https://arxiv.org/pdf/1512.03385>





Stride 2 & 필터수 2배 증가

다운샘플링과 채널 수가 변화될 때는
Skip-connection을 어떻게 해야 할까?

Stride=2를 사용하면 $F(x)$ 의 해상도가 줄어들어 x 보다 작아지기 때문에, residual 연결에서 두 값을 더하기 위해서는 x 도 적절하게 downsample(또는 변환)해야 합니다.

옵션 A (Zero-Padding을 통한 항등 Shortcut):

- $F(x)$ 의 채널 수가 증가하는 경우, x 의 채널 수가 부족하면 x 에 0을 패딩하여 $F(x)$ 와 동일한 차원으로 맞춥니다.
- 장점: 추가 학습 파라미터가 없으므로 간단하고 계산 비용이 낮습니다.
- 단점: 0으로 채워진 부분은 학습 가능한 정보가 없으므로, 경우에 따라 성능이 약간 떨어질 수 있습니다.

옵션 B (투영 Shortcut, Projection Shortcut):

- 1×1 컨볼루션을 사용하여 x 를 $F(x)$ 와 동일한 공간 및 채널 차원으로 변환합니다.
- 컨볼루션 연산은 stride=2를 사용하여 x 의 해상도를 downsample하며, 동시에 필요한 채널 수로 변환합니다.
- 장점: 학습 가능한 가중치를 사용하므로 x 를 더 정교하게 변환할 수 있어 정보 전달이 원활합니다.
- 단점: 추가 파라미터가 생기고 약간의 연산 비용이 늘어납니다.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112				$7 \times 7, 64, \text{stride } 2$	
conv2_x	56×56				$3 \times 3 \text{ max pool, stride } 2$	
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

18층과 34층까지 쌓았을 때의 실험 결과

1. 잔차학습을 적용하면 34층이 18층의 resnet 보다 우수한 결과
(이때 파라미터 개수는 동일)
2. 빠른 수렴 : 18층 plain과 Resnet은 정확도는 비슷하나 Resnet이 더 빠르게 수렴

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

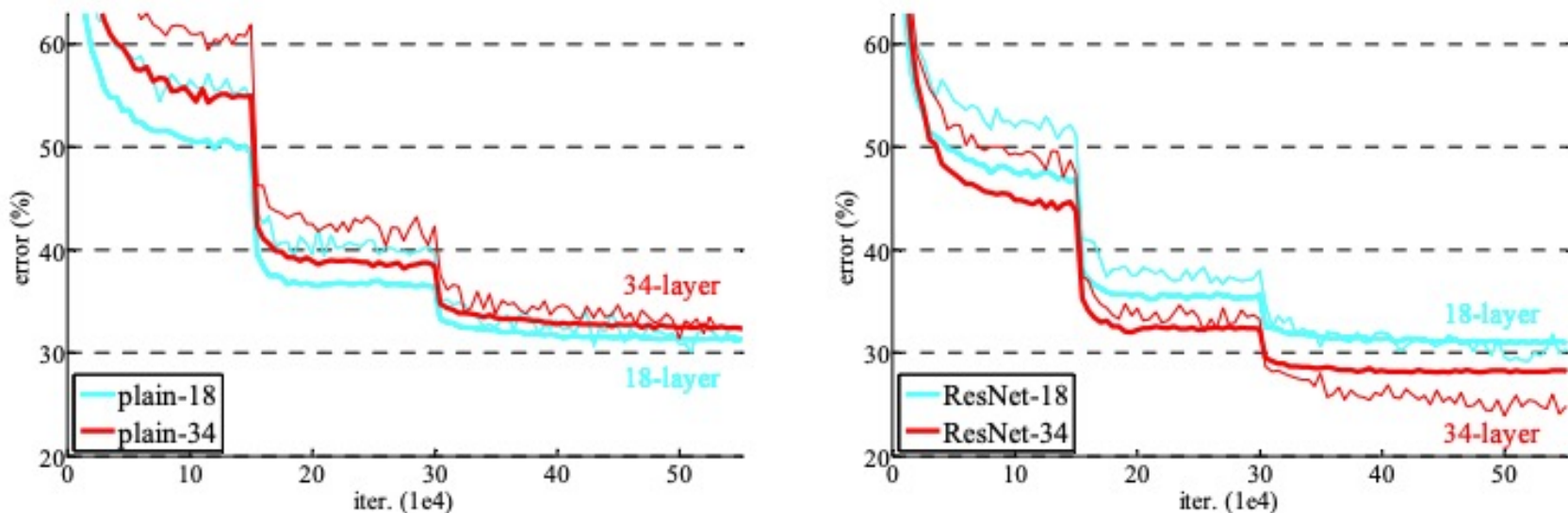


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

더 깊은 ResNet : Bottleneck 구조

- ImageNet을 위한 더 깊은 네트워크를 설명합니다.
- 학습 시간 문제로 인해, 우리는 기본 빌딩 블록을 bottleneck 설계로 수정합니다.
- 각 잔차 함수 FF에 대해 2층 대신 3층을 쌓으며, 이 세 층은 1×1 , 3×3 , 1×1 컨볼루션으로 구성됩니다.
- 여기서 1×1 레이어는 차원을 축소한 후 다시 복원하여, 3×3 레이어가 상대적으로 낮은 차원의 입력/출력을 가지는 bottleneck 역할을 하도록 합니다.

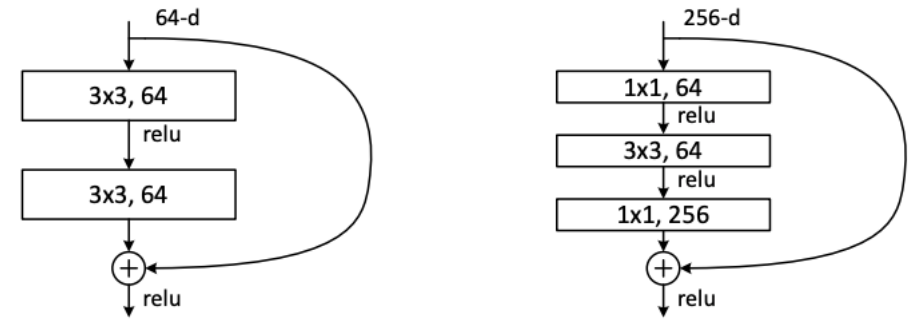


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	$7 \times 7, 64, \text{stride } 2$				
conv2_x	56×56	$3 \times 3 \text{ max pool, stride } 2$				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PRReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PRReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

1000층 이상 쌓는다면?

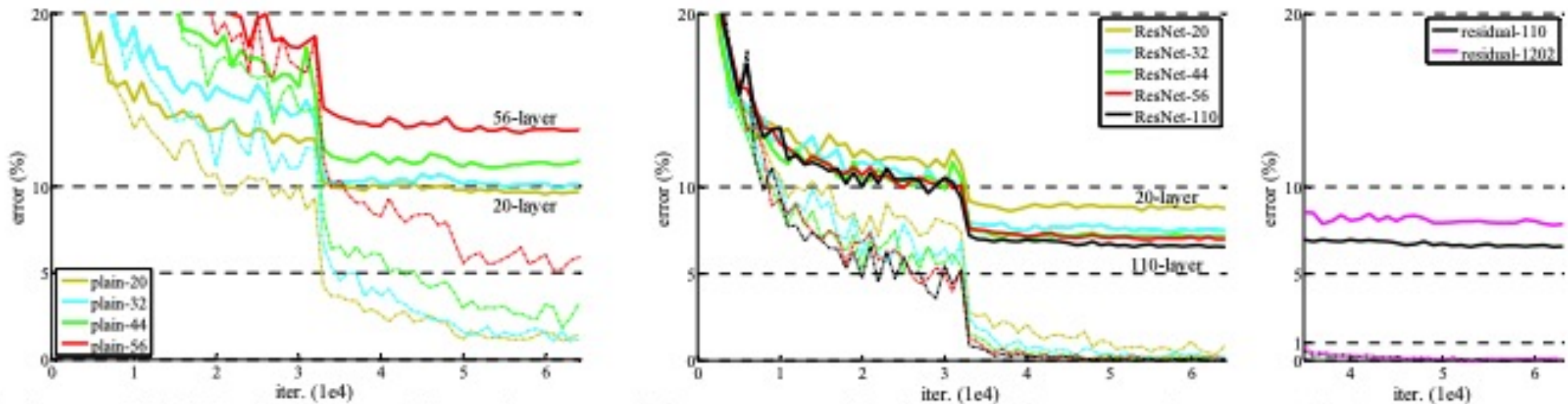
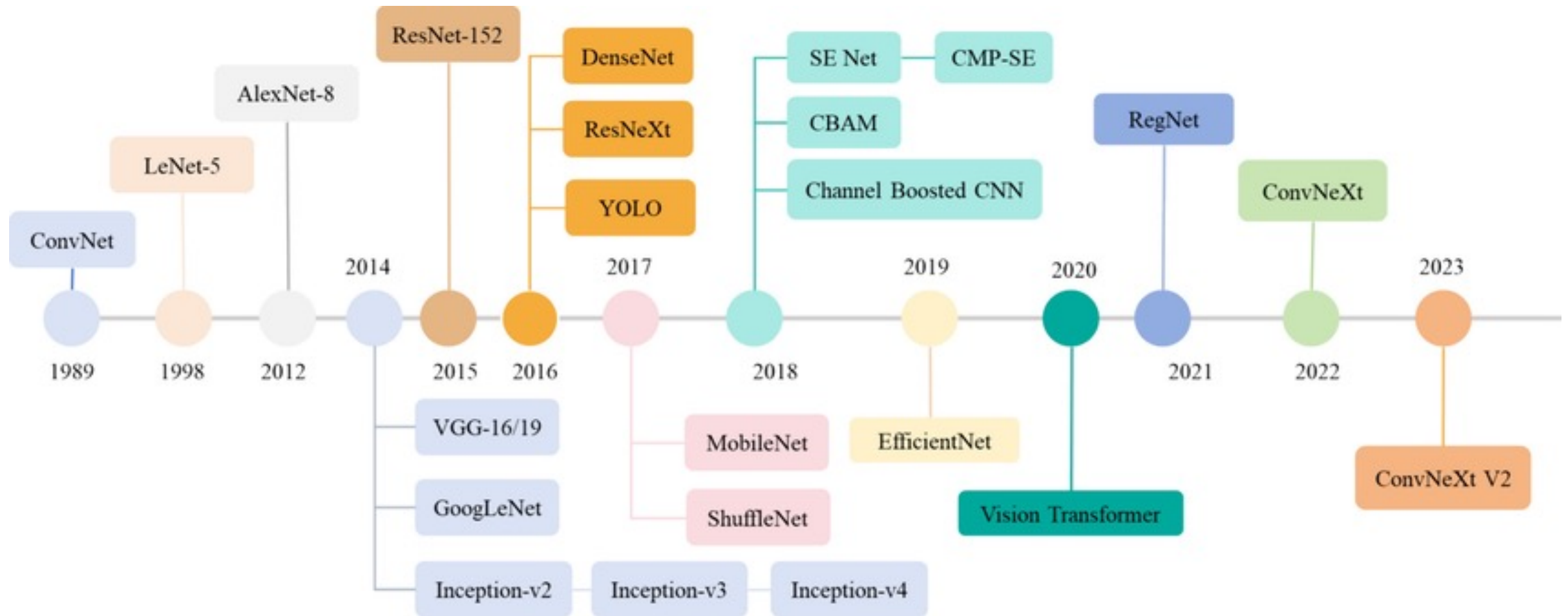


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left:** plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle:** ResNets. **Right:** ResNets with 110 and 1202 layers.

- ILSVRC(2015) 및 COCO(2015) 우승 모델
- 최초 결과(오차율 3.6%)가 인간 성능(5.1%)보다 우수
- 여러 버전 존재: 18, 34, 50, 101, 152 계층
- 각 합성곱(Conv) 계층마다 배치 정규화(Batch Normalization) 적용
- Xavier 초기화(Xavier Initialization) 사용
- SGD(확률적 경사 하강법) + 모멘텀 0.9 적용
- 초기 학습률 0.1 → 검증 오류가 정체되면 1/10로 감소
- 배치 크기(Batch size) : 256
- 가중치 감소(Weight Decay) 값: 1×10^{-5}
- 드롭아웃(Dropout) 사용 안 함



eXplainable AI (XAI)

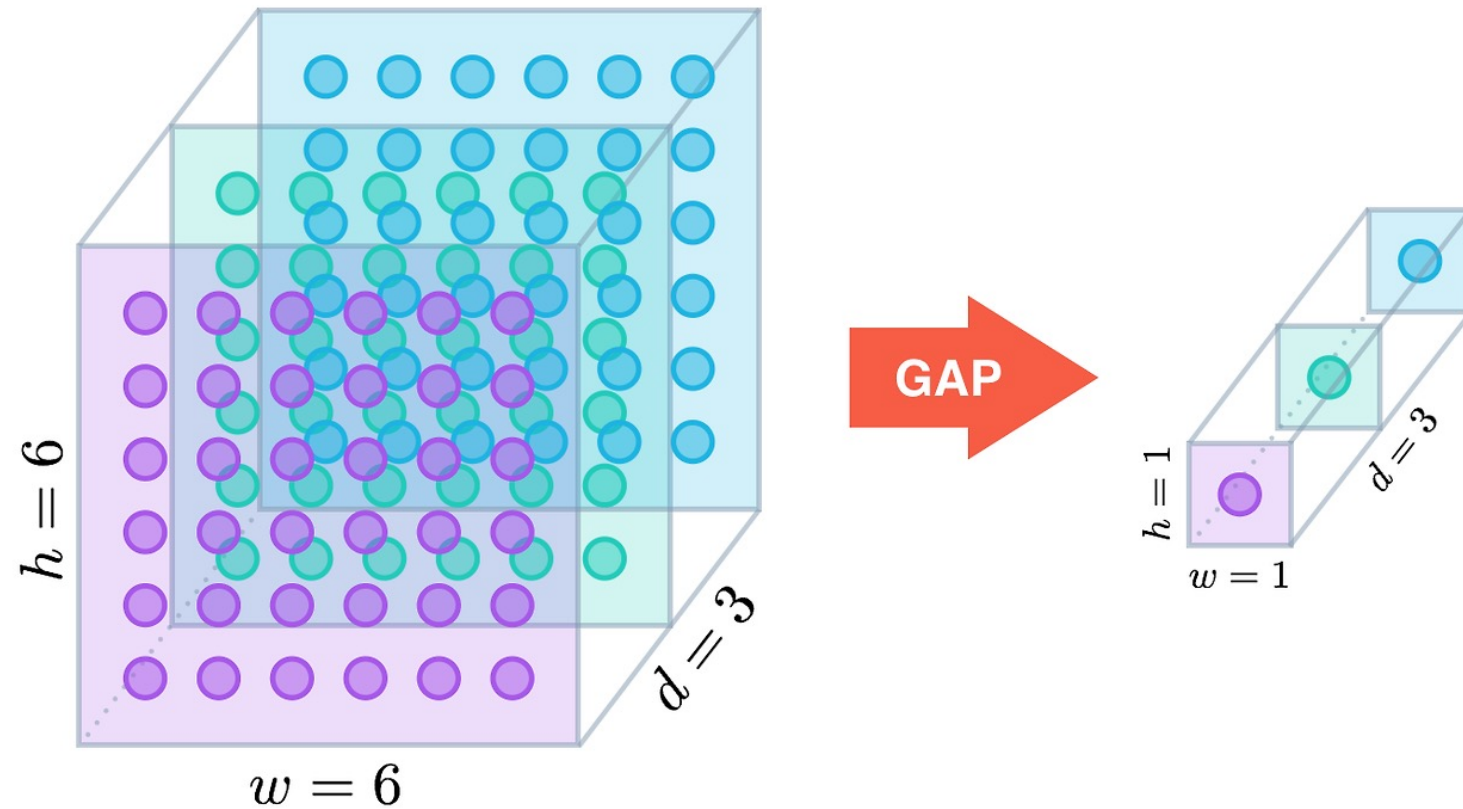
- XAI(설명 가능한 인공지능, eXplainable AI)는 인공지능 모델이 내린 결정이나 예측의 이유를 사람이 이해할 수 있도록 설명하는 기술과 방법을 의미합니다.
- 즉, 모델의 "블랙박스" 특성을 해소하여, 왜 특정 결과가 도출되었는지, 어떤 요인이 영향을 미쳤는지를 명확하게 보여줌으로써 신뢰성과 투명성을 높이는 것을 목표로 합니다
- **사후 해석(post-hoc interpretation):** 모델 학습 후, 예측 결과를 해석하는 방법
(예: CAM, Grad-CAM, LIME, SHAP 등)
- **모델 자체 해석(ante-hoc interpretability):** 처음부터 해석 가능한 모델을 설계하는 방법

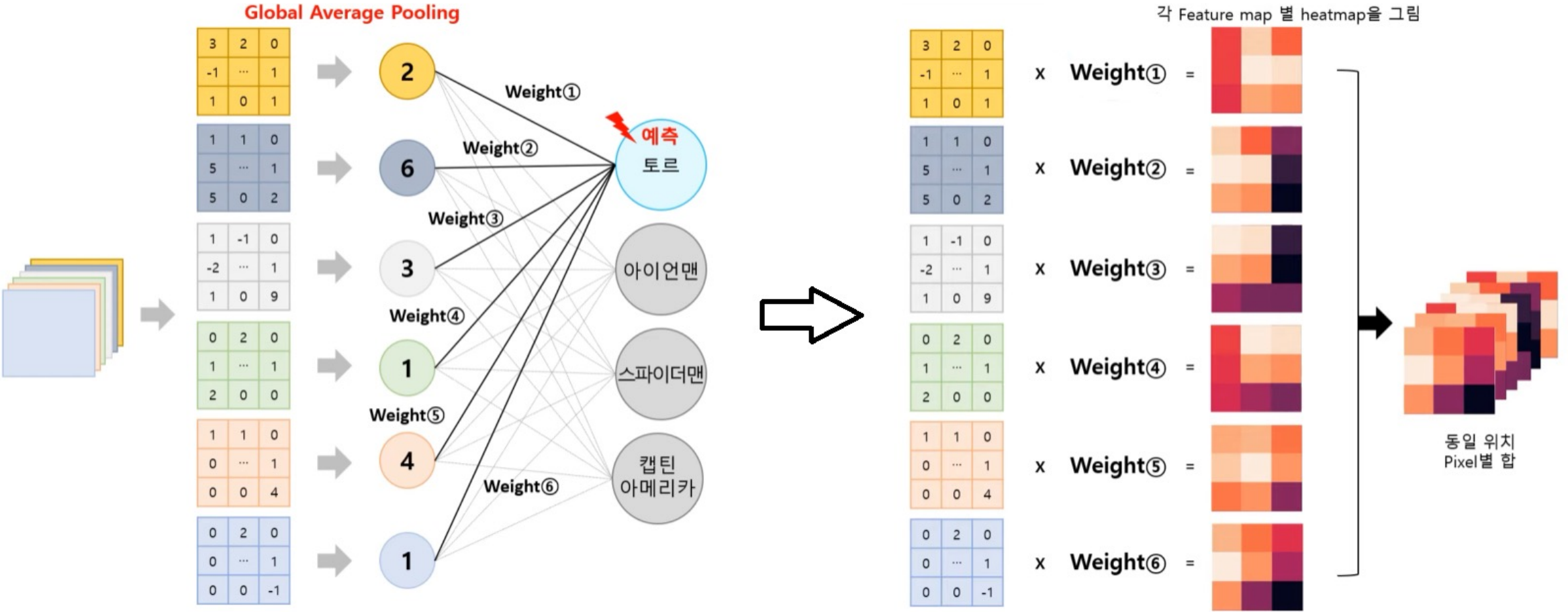
CAM (Class Activation Mapping)

- CAM(Class Activation Mapping)은 CNN 모델의 마지막 컨볼루션 계층을 활용하여 모델이 주목하는 영역을 시각화하는 기법입니다.
- 특정 클래스의 활성화 맵을 생성하여 모델의 예측 근거를 직관적으로 이해할 수 있습니다.
- 주요 기여
 - CNN의 마지막 컨볼루션 계층을 활용하여 특정 클래스가 활성화되는 영역을 직접 시각화할 수 있습니다.
 - 이미지 분류 및 물체 탐지에서 모델의 해석 가능성을 높이는 데 기여하였습니다.
 - Explainable AI(XAI) 연구의 중요한 기초 기술로 활용됩니다.
-  논문: <https://arxiv.org/pdf/1512.04150>

Global Average Pooling(GAP)

- i 번째 feature map의 평균값을 구해서 i 번째 출력 노드에 입력하는 것





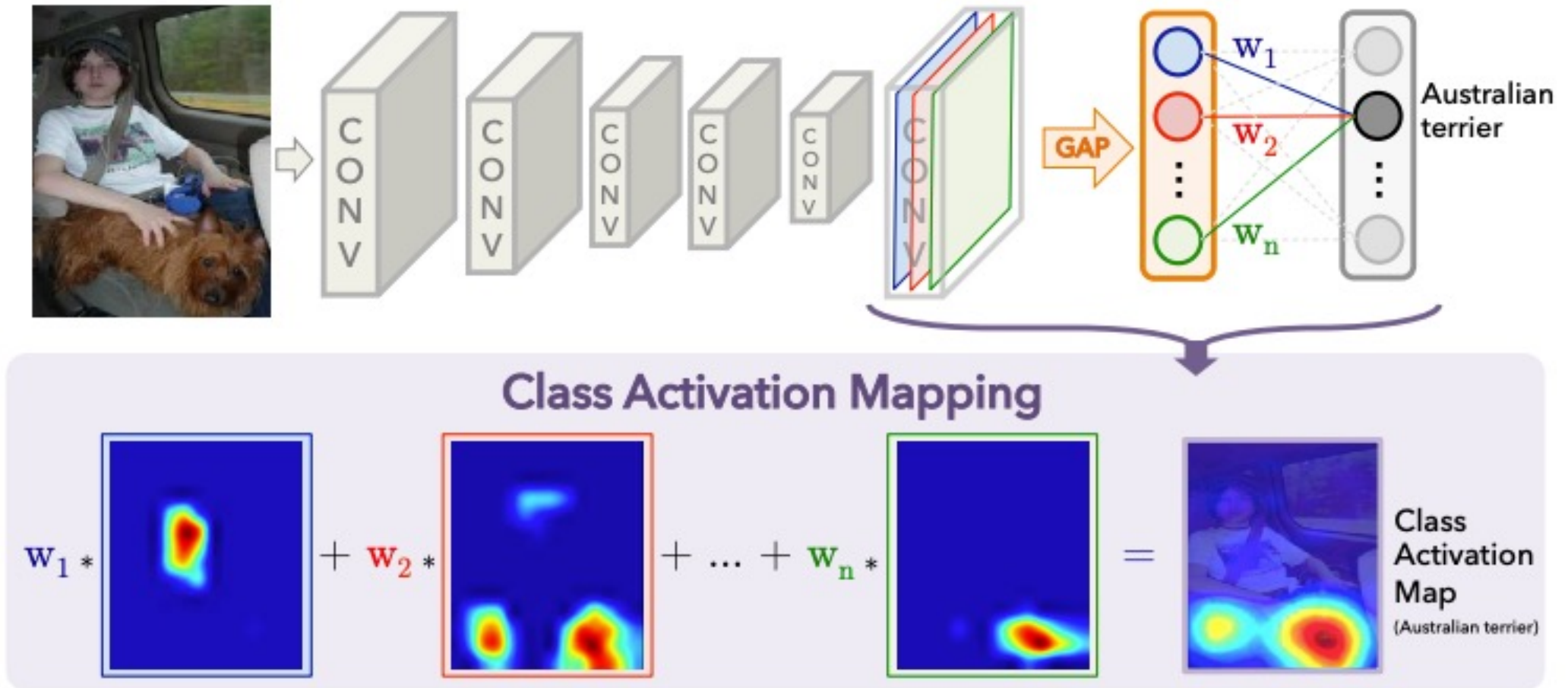


Figure 2. Class Activation Mapping: the predicted class score is mapped back to the previous convolutional layer to generate the class activation maps (CAMs). The CAM highlights the class-specific discriminative regions.

1. **컨볼루션 층**: 입력 이미지를 여러 컨볼루션 층을 거쳐 feature map $f_k(x, y)$ 를 생성합니다.
2. **GAP 적용**: 각 feature map f_k 의 모든 위치에서 평균을 취하여 하나의 값 F_k 를 얻습니다.
3. **클래스 가중치**: 학습 과정에서 각 클래스 c 에 대해 학습된 가중치 w_k^c 가 존재합니다.
4. **클래스 점수 계산**: 각 클래스의 점수는 다음과 같이 계산됩니다.

$$S^c = \sum_k w_k^c \cdot F_k$$

5. **CAM 생성**: 다시 공간 정보로 되돌리기 위해, 각 위치 (x, y) 에 대해 다음과 같이 계산합니다.

$$M_c(x, y) = \sum_k w_k^c \cdot f_k(x, y)$$

이 맵 $M_c(x, y)$ 가 높을수록 해당 영역이 클래스 c 에 중요한 역할을 했음을 의미합니다.



장점

- 단순한 구조 변경(GAP 도입)만으로 해석 가능.
- 모델의 결정 과정에서 어떤 영역이 기여했는지 직관적으로 확인할 수 있음.

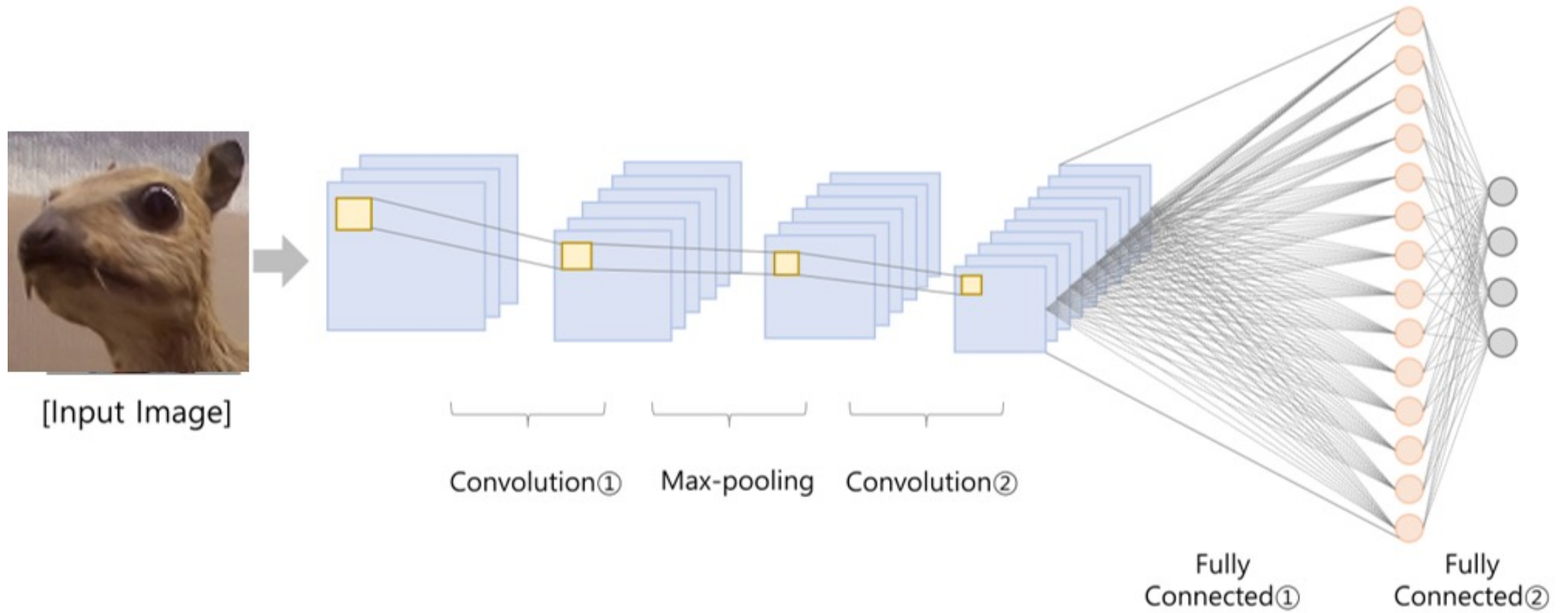
제한 사항

- 네트워크 구조에 GAP가 반드시 포함되어야 함.
즉, FCN을 사용하는 모델에는 바로 적용하기 어려움.
- 모델 구조에 제약을 주어야 하기 때문에 기존 모델의 아키텍처 변경이 불가피함.

Grad-CAM (Gradient-weighted Class Activation Mapping)

- Grad-CAM은 Gradient 정보를 활용하여 모델이 특정 예측을 할 때 중요하게 본 영역을 강조하는 기법
- 기존 CAM의 단점을 개선하여 사전 학습된 다양한 CNN 모델에도 적용 가능합니다.
- 주요 기여
 - Gradient 정보를 이용하여 CNN 모델이 특정 클래스에 대해 어떤 부분을 주목하는지를 효과적으로 시각화할 수 있습니다.
 - 사전 학습된 다양한 CNN 아키텍처(ResNet, VGG 등)에도 적용 가능하여 활용 범위가 더욱 넓어졌습니다.
 - 의료 영상 분석, 자율 주행 등 설명 가능한 AI(Explainable AI, XAI) 분야에서 널리 사용됩니다.
- 논문: <https://arxiv.org/pdf/1610.02391>

Grad-CAM (Gradient-weighted Class Activation Mapping)



동작 원리

1. 순전파 단계:

- 입력 이미지에 대해 네트워크를 순전파하여 클래스 점수 y^c 를 계산합니다.

2. 역전파 단계:

- 선택한 클래스 c 의 점수에 대해, 마지막 컨볼루션 층의 feature map A^k 에 대한 **gradient** $\frac{\partial y^c}{\partial A_{ij}^k}$ 를 계산합니다.

3. 가중치 계산:

- 각 feature map k 에 대해, gradient의 공간적 평균을 구하여 가중치 α_k^c 를 산출합니다.

$$\alpha_k^c = \overbrace{\frac{1}{Z} \sum_i \sum_j}^{\text{global average pooling}} \underbrace{\frac{\partial y^c}{\partial A_{ij}^k}}_{\text{gradients via backprop}}$$

* 여기서 Z는 feature map의 픽셀 수 (예: $Z=H \times W$)

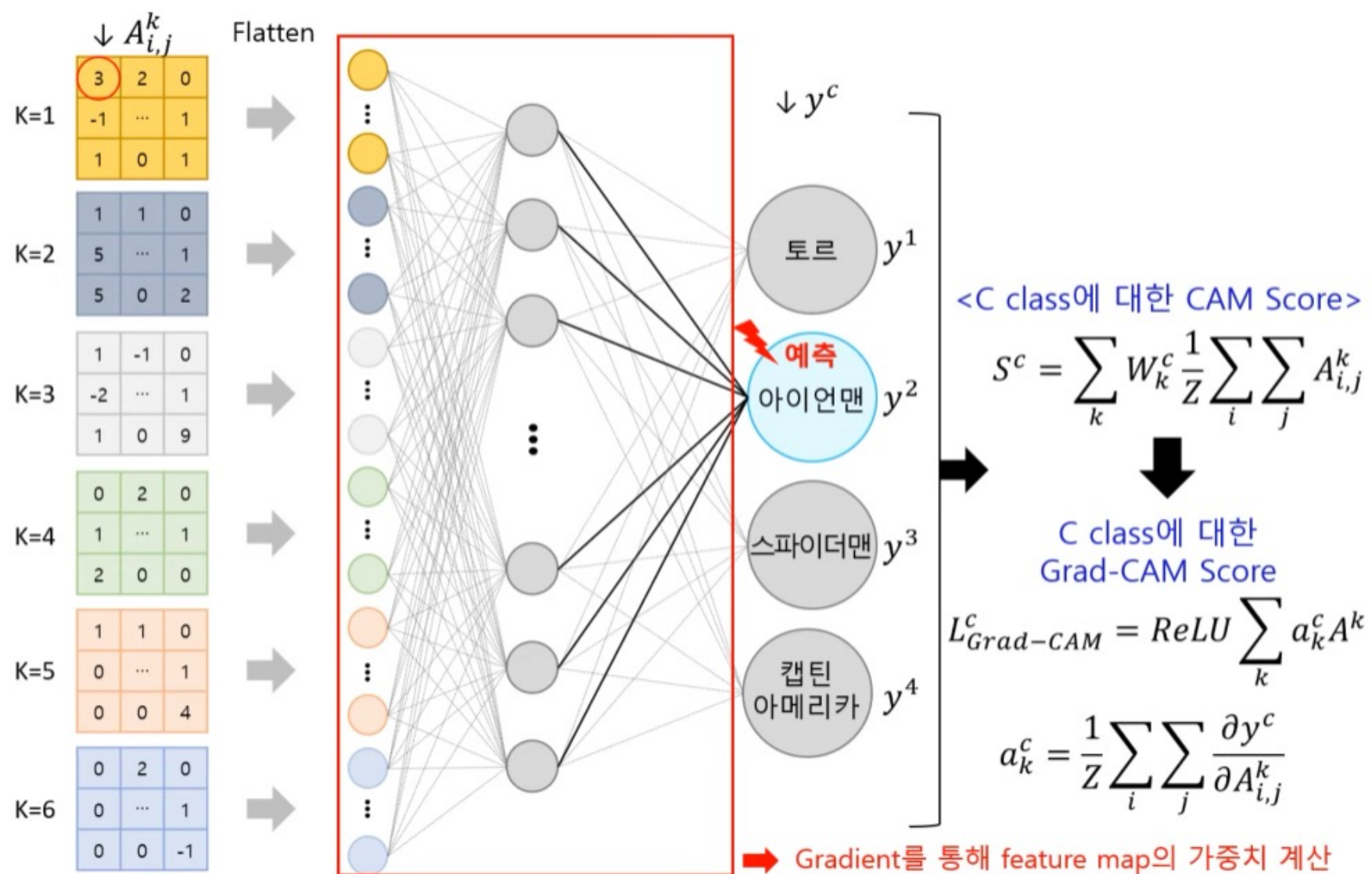
동작 원리

4. Grad-CAM 맵 생성:

- 각 feature map A^k 에 대해 위에서 구한 가중치 α_k^c 를 곱한 후 합산하고, ReLU를 적용하여 최종 시각화 맵 $L_{\text{Grad-CAM}}^c$ 를 생성합니다.

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right)$$

- **ReLU 적용 이유:** 음수 값은 해당 클래스와의 긍정적 관계를 나타내지 않으므로, 관심 영역을 강조하기 위해 음수를 0으로 만듭니다.



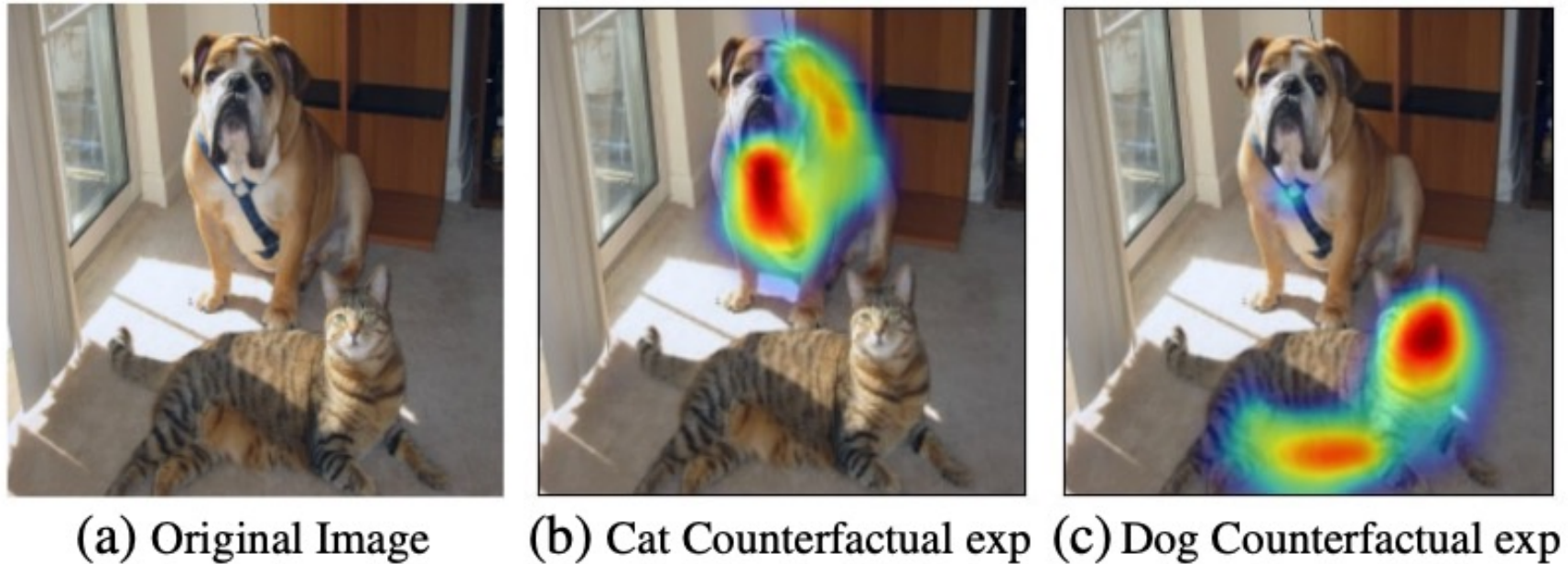


Fig. 3: Counterfactual Explanations with Grad-CAM

장점

- **모델 구조 무관:** Grad-CAM은 모델의 마지막 컨볼루션 층만 있으면 적용 가능하므로, GAP가 없는 다양한 CNN 아키텍처에 사용할 수 있습니다.
- **직관적 시각화:** 계산된 Grad-CAM 맵은 이미지의 어떤 영역이 해당 클래스 예측에 기여했는지 직관적으로 보여주어, 모델의 결정 과정을 이해하는 데 도움이 됩니다.
- **디버깅 및 개선 도구:** 예측 오류나 의도하지 않은 편향을 가진 영역을 식별함으로써, 모델 개선 및 학습 데이터 보완에 활용할 수 있습니다.

제한 사항

- **모델 출력의 영향을 받음:** 역전파를 통한 gradient 계산이므로, 모델의 출력 스케일이나 비선형 활성화 함수의 특성에 따라 Grad-CAM 맵이 다르게 나타날 수 있습니다.
- **노이즈에 민감:** 일부 경우에 gradient가 노이즈를 포함할 수 있어, 시각화 결과가 불안정하게 나타날 가능성이 있습니다.
- **양의 영향만 고려:** ReLU 적용으로 인해 음수 기여는 모두 제거되는데, 이로 인해 부정적 기여(어떤 영역이 예측을 억제한 경우)에 대한 정보는 제공되지 않습니다.

항목	CAM	Grad-CAM
구조 의존성	GAP 등 특정 구조(마지막에 GAP를 적용한 모델)에 한정됨	네트워크 구조에 구애받지 않음
해석 방법	학습된 가중치와 feature map의 선형 조합	역전파로 얻은 gradient의 공간적 평균값을 가중치로 활용
적용 범위	제한적: 전결합층 대신 GAP가 사용된 모델에서만 적용 가능	범용적: 다양한 CNN 아키텍처 및 태스크에 적용 가능
계산 복잡도	상대적으로 단순	역전파를 추가로 계산해야 하므로 약간의 추가 연산 필요

감사합니다 😊